

Aula 09

*DataPrev (Perfil 3: Desenvolvimento de Software) Engenharia de Software - 2026
(Pós-Edital)*

Autor:

**Diego Carvalho, Equipe
Informática 2 (Diego Carvalho),
Equipe Informática 1 (Diego
Carvalho)**

09 de Julho de 2026

Índice

1) Web Services - Teoria	3
2) Web Services - Questões Comentadas - FGV	38
3) Web Services - Lista de Questões - FGV	51

WEB SERVICES

Conceitos Fundamentais

INCIDÊNCIA EM PROVA: ALTÍSSIMA

O crescimento das organizações levou à adoção de sistemas especializados para diferentes áreas, como vendas, finanças e recursos humanos. Esse cenário trouxe um desafio: sem integração adequada, os dados ficam isolados em ambientes distintos, exigindo transferências manuais. Como consequência, aumentam os custos operacionais e a probabilidade de erros. Integrações desenvolvidas individualmente tornam o ambiente mais complexo, dificultando atualizações e gerando dependências que complicam alterações nas aplicações já existentes.

A duplicação de informações representa um desafio relevante nos ambientes corporativos. Quando a troca de dados ocorre de forma limitada, cada sistema tende a armazenar sua própria versão das informações, gerando inconsistências e dificultando auditorias. Para superar essas dificuldades, foram desenvolvidos mecanismos padronizados de integração. A padronização simplifica a troca de informações, aumenta a confiabilidade das interações e permite que sistemas compartilhem dados sem precisar conhecer detalhes da implementação interna de cada aplicação.

Serviços Web

INCIDÊNCIA EM PROVA: ALTA

SERVIÇOS WEB

Serviços Web, ou Web Services, são mecanismos que permitem a comunicação entre aplicações por meio de uma rede utilizando protocolos e formatos padronizados. Seu principal objetivo é disponibilizar funcionalidades ou informações para outros sistemas de forma independente da tecnologia utilizada em sua implementação. Dessa forma, diferentes aplicações podem consumir serviços remotos sem precisar conhecer detalhes internos de funcionamento do sistema provedor.

VISÃO GERAL DOS SERVIÇOS WEB (WEB SERVICES)



Os Web Services surgiram como solução para permitir que aplicações diferentes se comuniquem por redes utilizando padrões amplamente aceitos. Sua principal finalidade é disponibilizar funcionalidades para outros sistemas de forma padronizada, independentemente das tecnologias utilizadas em sua construção. A comunicação ocorre por meio de interfaces e contratos previamente definidos, que estabelecem quais operações podem ser executadas, quais dados devem ser enviados e quais respostas são retornadas, sem exigir acesso à implementação interna do provedor.

Uma característica central dos Web Services é o desacoplamento entre fornecedor e consumidor. O sistema consumidor não precisa conhecer linguagem, banco de dados ou infraestrutura do provedor – basta compreender as regras do contrato. Entre as abordagens mais conhecidas estão SOAP e REST. Seriam a mesma coisa? Não. SOAP é um protocolo baseado em padrões específicos de comunicação, enquanto REST é um estilo arquitetural. Ambas, porém, têm como objetivo facilitar a comunicação entre aplicações em ambientes heterogêneos.

Componentes Principais

Os Web Services dependem de diversos elementos para que a comunicação entre aplicações distribuídas ocorra de forma organizada. Quando um sistema precisa utilizar funcionalidades de outro, surgem questões essenciais: como localizar o serviço, quais informações trocar e quais regras seguir? Esses aspectos são respondidos por diferentes elementos arquiteturais que viabilizam a integração entre sistemas construídos com tecnologias distintas, com responsabilidades bem definidas para localização, operações, protocolos e formatos de dados.

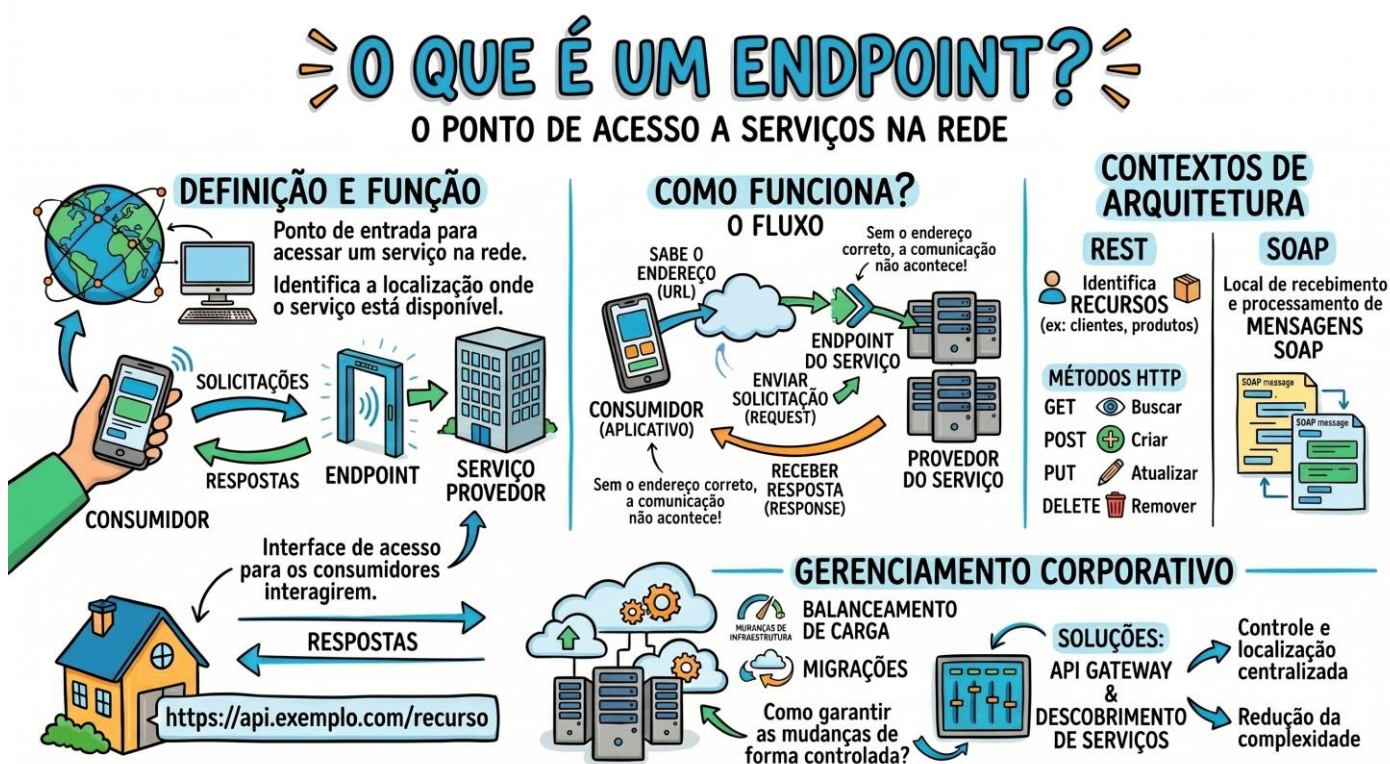
A separação de responsabilidades torna a comunicação organizada e previsível, facilitando a interoperabilidade entre aplicações de equipes diferentes. Um consumidor não precisa conhecer os detalhes internos do serviço – basta compreender a interface disponibilizada. Alterações internas podem ocorrer sem impactar consumidores, desde que contratos permaneçam compatíveis. Os principais elementos presentes em soluções baseadas em serviços são os endpoints, as operações, as mensagens, os contratos, os protocolos e os formatos de dados.

Endpoint

INCIDÊNCIA EM PROVA: MÉDIA

ENDPOINT

Um endpoint é o endereço por meio do qual um serviço pode ser acessado na rede. Normalmente representado por uma URL, ele identifica o ponto de entrada para envio de requisições e recebimento de respostas. Em APIs REST, por exemplo, diferentes endpoints podem representar diferentes recursos do sistema. O endpoint funciona como a interface de acesso utilizada pelos consumidores para interagir com os serviços disponibilizados.



O endpoint é um dos elementos mais importantes de um Web Service, pois identifica o local onde o serviço pode ser acessado na rede. Ele corresponde ao ponto de comunicação utilizado pelos

consumidores para enviar requisições e receber respostas, normalmente representado por uma URL. Como uma aplicação sabe para onde encaminhar uma solicitação? Essa informação é fornecida pelo endpoint. Sem ele, mesmo que o serviço estivesse funcionando perfeitamente, não haveria como encaminhar requisições ao destino adequado na rede.

A forma de utilizar os endpoints varia conforme a arquitetura adotada. Em ambientes REST, eles identificam recursos ou coleções, como clientes e produtos, e as operações são realizadas por métodos HTTP como GET, POST, PUT e DELETE. Em implementações SOAP, o endpoint indica onde o serviço recebe e processa mensagens. Em ambientes corporativos, gateways de APIs e soluções de descoberta de serviços facilitam o gerenciamento dos endpoints disponíveis, reduzindo a complexidade operacional do ambiente de integração distribuída.

Operações

INCIDÊNCIA EM PROVA: BAIXA

OPERAÇÕES

Operações representam as funcionalidades que um Web Service disponibiliza aos seus consumidores. Cada operação define uma ação específica que pode ser executada, como consultar dados, cadastrar informações, atualizar registros ou remover recursos. Em serviços SOAP, as operações são normalmente descritas em contratos WSDL. Em APIs REST, as operações geralmente são associadas aos métodos HTTP, como GET, POST, PUT e DELETE.

As operações correspondem às funcionalidades que um Web Service disponibiliza para outras aplicações. Sempre que um consumidor interage com um serviço, existe um objetivo específico: consultar informações, cadastrar registros, atualizar dados ou remover recursos. A resposta sobre o que um sistema consegue fazer ao acessar determinado serviço está nas operações disponibilizadas. Elas são invocadas quando o consumidor envia uma requisição com os dados necessários e recebe informações compatíveis com a solicitação realizada ao provedor.



A forma de representar operações varia conforme a arquitetura. Em SOAP, elas são definidas explicitamente no contrato, geralmente por meio de WSDL. Em REST, o foco está nos recursos, e as ações são executadas pelos métodos HTTP: GET, POST, PUT e DELETE. O projeto adequado das operações influencia a qualidade do serviço. Operações bem estruturadas reduzem ambiguidades, facilitam o entendimento pelos consumidores e tornam a interface previsível, permitindo que sistemas utilizem os recursos de forma padronizada e eficiente.

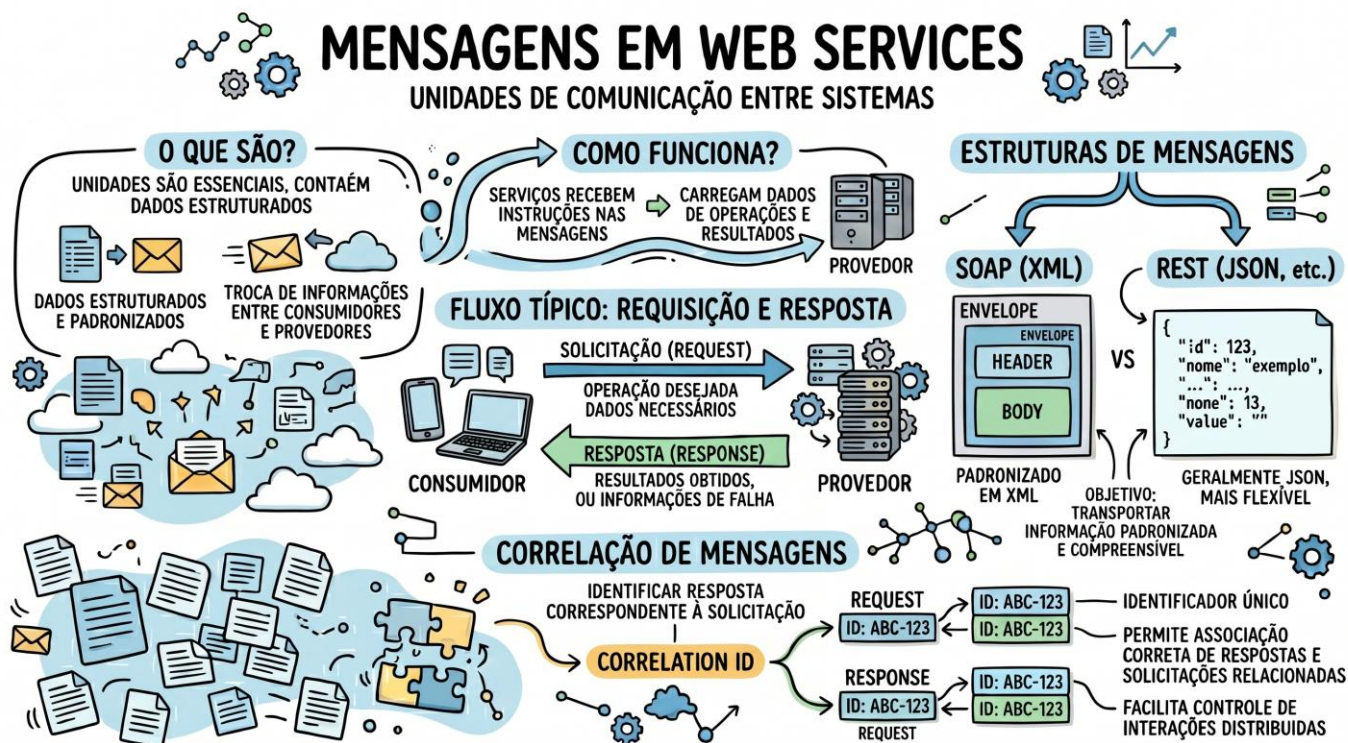
Mensagens

INCIDÊNCIA EM PROVA: BAIXA

MENSAGENS

Mensagens são as unidades de comunicação trocadas entre consumidores e provedores de serviços. Elas contêm os dados enviados em requisições e respostas, seguindo formatos e estruturas previamente definidos. Em SOAP, as mensagens são organizadas em documentos XML padronizados. Em REST, é comum a utilização de formatos como JSON. As mensagens permitem que informações sejam transmitidas de forma estruturada e compreensível por diferentes sistemas.

As mensagens constituem o principal mecanismo de comunicação entre consumidores e provedores em Web Services. Toda interação ocorre por envio e recebimento de mensagens que transportam dados relacionados às operações solicitadas. O fluxo envolve dois tipos principais: requisições, contendo a operação desejada e os dados necessários, e respostas, retornadas após o processamento com os resultados ou informações sobre falhas. A estrutura dessas mensagens varia conforme a tecnologia utilizada – SOAP usa XML padronizado; REST usa normalmente JSON.



Em arquiteturas SOAP, as mensagens seguem formato padronizado em XML, organizado em Envelope, Header e Body. Em serviços REST, são normalmente utilizados JSON ou XML. Outro conceito associado à troca de mensagens é a Message Correlation. Como identificar qual resposta corresponde a qual solicitação quando várias mensagens são processadas simultaneamente? Utiliza-se um identificador único

denominado Correlation ID, que acompanha a comunicação e permite associar corretamente respostas a requisições em integrações assíncronas entre sistemas.

Contratos

INCIDÊNCIA EM PROVA: BAIXA

CONTRATOS

Contratos são documentos ou especificações que definem as regras de interação entre consumidores e provedores de serviços. Eles descrevem quais operações estão disponíveis, quais parâmetros devem ser fornecidos, quais formatos de dados serão utilizados e quais respostas podem ser esperadas. O contrato atua como um acordo formal de comunicação, permitindo que diferentes sistemas interajam de maneira previsível e padronizada.

O contrato de serviço define formalmente como a comunicação entre consumidores e provedores deve ocorrer. Ele estabelece operações disponíveis, dados aceitos nas requisições, formatos das respostas e outras regras que orientam a interação. Um consumidor precisa conhecer detalhes internos do serviço? Não – basta compreender as regras do contrato. Essa separação entre interface e implementação reduz o acoplamento: alterações internas podem ocorrer sem impactar consumidores, desde que o contrato permaneça compatível com as expectativas já estabelecidas.



A forma de representar o contrato varia conforme a tecnologia. Em SOAP, ele é formalizado por meio do WSDL, que descreve operações, mensagens, tipos de dados e pontos de acesso. Em REST, não há exigência de contrato formal equivalente, embora especificações como OpenAPI sejam amplamente utilizadas. Contratos bem definidos são fundamentais para as integrações: quando claros e documentados, reduzem ambiguidades no desenvolvimento, facilitam a manutenção e ampliam a capacidade de reutilização dos serviços em diferentes contextos.

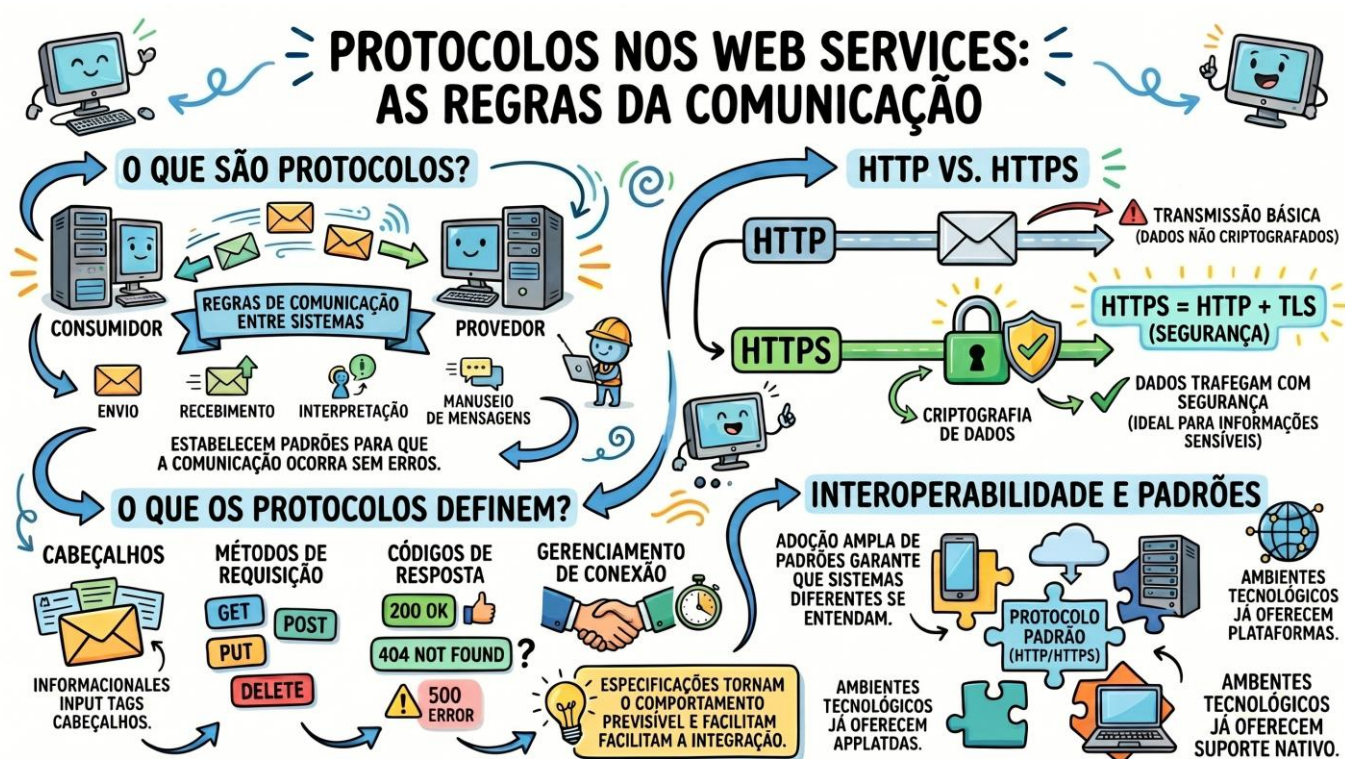
Protocolos

INCIDÊNCIA EM PROVA: BAIXA

PROTOCOLOS

Protocolos são conjuntos de regras que definem como a comunicação entre sistemas deve ocorrer. Eles estabelecem procedimentos relacionados ao envio, recebimento, interpretação e tratamento das mensagens trocadas. Nos Web Services, protocolos como HTTP e HTTPS são amplamente utilizados para transportar dados entre consumidores e provedores. A utilização de protocolos padronizados é fundamental para garantir interoperabilidade entre diferentes plataformas.

Os protocolos definem as regras que permitem a comunicação entre consumidores e provedores em Web Services. Sempre que duas aplicações trocam informações, ambas precisam seguir procedimentos compatíveis para envio, recebimento e interpretação das mensagens. Entre os mais utilizados estão HTTP e HTTPS. Qual a diferença? O HTTP fornece os mecanismos básicos para troca de informações em rede. Já o HTTPS utiliza o HTTP combinado com o TLS, permitindo que dados trafeguem de forma protegida por criptografia entre os sistemas.



Os protocolos também definem aspectos como cabeçalhos, métodos de requisição, códigos de resposta e mecanismos de gerenciamento de conexões, tornando o comportamento da comunicação previsível. A ampla adoção de protocolos padronizados favorece a interoperabilidade dos Web Services. Por que sistemas construídos em plataformas diferentes conseguem se comunicar com relativa facilidade? Porque a maioria dos ambientes tecnológicos oferece suporte nativo a protocolos amplamente utilizados, estabelecendo comunicação consistente entre as aplicações distribuídas.

Formatos de Dados

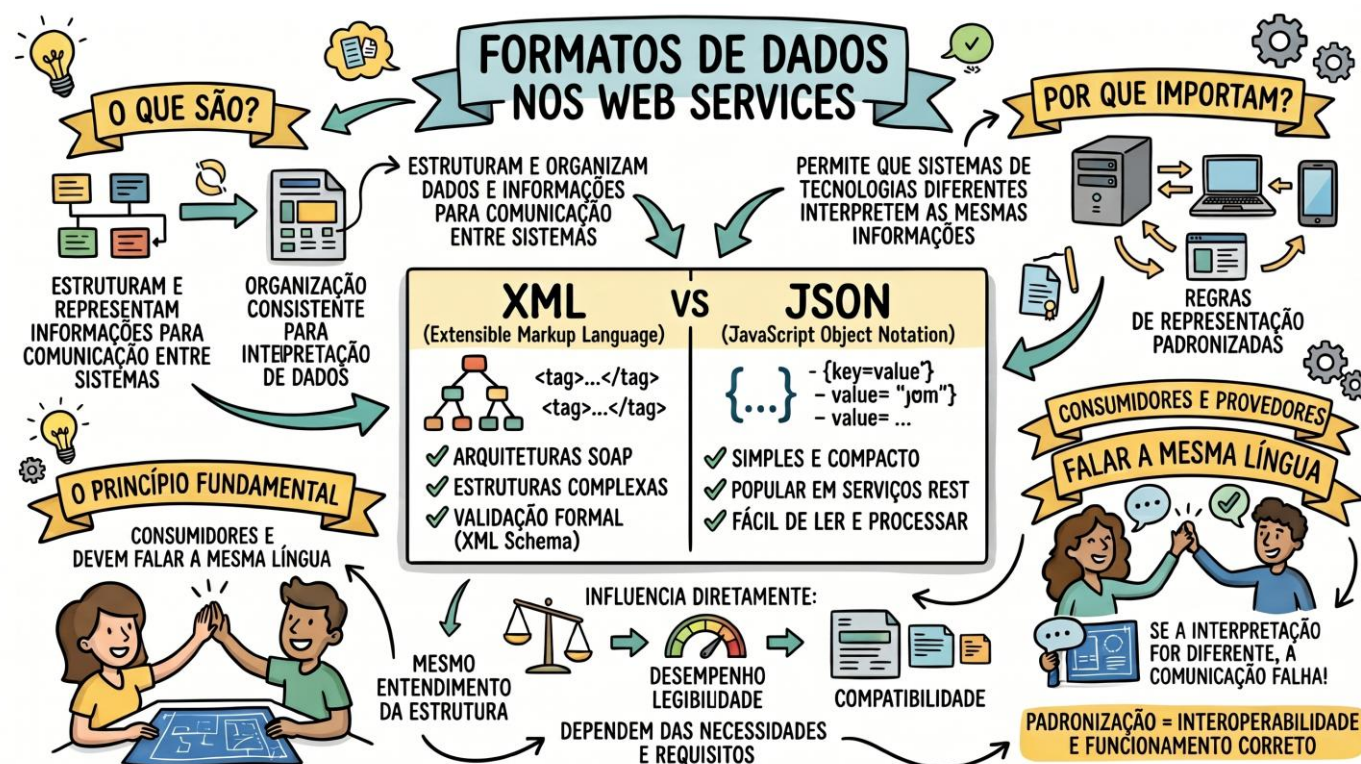
INCIDÊNCIA EM PROVA: BAIXA

FORMATOS DE DADOS

Formatos de dados definem a maneira como as informações são estruturadas e representadas durante a comunicação entre sistemas. Nos Web Services, os formatos mais comuns são XML e JSON. Esses padrões permitem organizar informações de maneira consistente, facilitando sua interpretação por aplicações distintas. A

escolha do formato influencia aspectos como legibilidade, tamanho das mensagens, desempenho e compatibilidade entre tecnologias.

Os formatos de dados definem como as informações serão organizadas dentro das mensagens trocadas entre consumidores e provedores. Entre os mais utilizados em Web Services destacam-se XML e JSON. O XML foi amplamente adotado em arquiteturas SOAP pela capacidade de representar estruturas complexas e oferecer validação via XML Schema. O JSON surgiu como alternativa mais simples e compacta. Por que se tornou tão popular em serviços REST? Porque sua estrutura é mais fácil de ler, processar e transmitir em muitos cenários de integração.



A escolha do formato influencia a comunicação entre sistemas. Alguns produzem mensagens menores, reduzindo o volume de dados transmitidos. Outros oferecem recursos mais sofisticados para validação e representação de relacionamentos complexos. O que aconteceria se cada sistema interpretasse uma mensagem de maneira diferente? A comunicação deixaria de ser confiável e os resultados se tornariam inconsistentes. Por essa razão, consumidores e provedores precisam compartilhar o mesmo entendimento sobre a estrutura dos dados trocados nas integrações.

HTTP/HTTPS

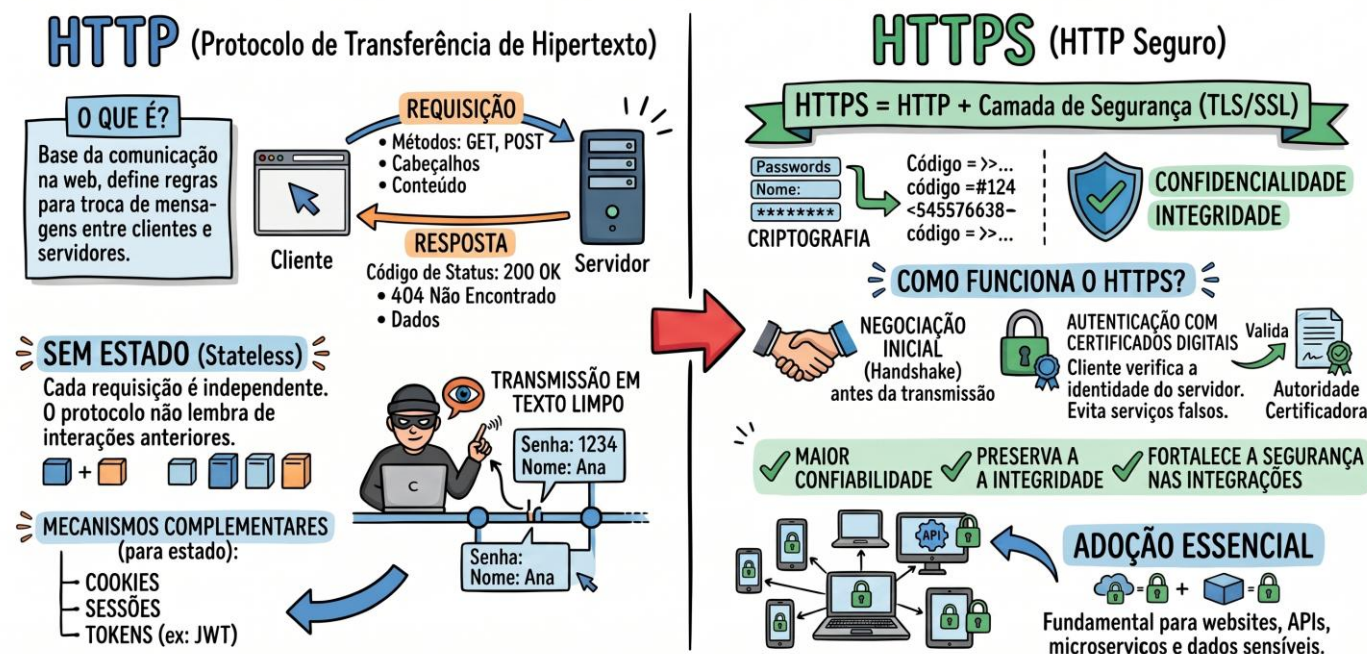
INCIDÊNCIA EM PROVA: MÉDIA

HTTP/HTTPS

HTTP é o protocolo mais utilizado para transportar mensagens entre consumidores e provedores de serviços na Web. Ele define métodos, cabeçalhos e mecanismos de comunicação que permitem a troca de informações entre aplicações. HTTPS corresponde à versão segura do HTTP, adicionando criptografia por meio de TLS ou SSL. Dessa forma, garante confidencialidade, integridade e autenticidade das informações transmitidas pela rede.

O HTTP, Hypertext Transfer Protocol, é responsável por grande parte da comunicação realizada na Internet. Seu funcionamento baseia-se no modelo requisição-resposta: o cliente envia uma solicitação indicando o recurso desejado, enquanto o servidor processa e retorna uma resposta. As mensagens podem incluir métodos de requisição, cabeçalhos, parâmetros e conteúdos adicionais. Uma característica importante do HTTP é seu comportamento stateless: cada requisição é tratada de forma independente, sem manter informações sobre interações anteriores.

HTTP x HTTPS: O PROTOCOLO DA WEB E SUA VERSÃO SEGURA



Quando uma aplicação precisa preservar dados entre solicitações, utiliza mecanismos complementares como cookies, sessões ou tokens de autenticação. O comportamento stateless favorece a simplicidade e a escalabilidade do protocolo. Diante da necessidade de proteger dados transmitidos, foi desenvolvido o HTTPS, que combina HTTP com o TLS para criptografar as informações. Antes da transmissão, cliente e servidor realizam uma negociação para estabelecer conexão segura, dificultando a interceptação e alteração indevida dos dados durante o percurso.

Além da criptografia, o HTTPS oferece mecanismos de autenticação baseados em certificados digitais. Qual a utilidade desses certificados? Eles permitem que o cliente verifique a identidade do servidor, reduzindo riscos de ataques que redirecionam usuários para serviços falsos. Essa verificação é especialmente importante em aplicações que manipulam credenciais de acesso, dados pessoais ou

informações financeiras sensíveis. A combinação da criptografia TLS com a autenticação por certificados preserva a integridade das comunicações realizadas pela Web.

A adoção do HTTPS tornou-se uma prática essencial no desenvolvimento de aplicações modernas. Navegadores e plataformas da Internet consideram conexões seguras um requisito fundamental. Por que a utilização de certificados digitais é tão relevante? Porque garantem a autenticidade do servidor, reduzindo vulnerabilidades nas comunicações. Como consequência, aplicações protegidas por HTTPS oferecem maior confiabilidade, preservam a integridade dos dados transmitidos e fortalecem a segurança das integrações realizadas entre sistemas distribuídos.

■

Web Services SOAP

Conceitos Básicos

INCIDÊNCIA EM PROVA: ALTÍSSIMA

SOAP

SOAP é um protocolo baseado em XML utilizado para troca estruturada de mensagens entre aplicações. Seu principal objetivo é fornecer um padrão rigoroso de comunicação, independente das tecnologias utilizadas pelos sistemas participantes. O SOAP define uma estrutura formal para as mensagens e pode ser complementado por diversas especificações relacionadas à segurança, confiabilidade e transações, sendo amplamente utilizado em ambientes corporativos.

SOAP, Simple Object Access Protocol, é um protocolo de comunicação desenvolvido para a troca estruturada de informações entre aplicações distribuídas. Em ambientes corporativos, sistemas construídos com plataformas diferentes precisam compartilhar dados de forma confiável. Como garantir que todos se comuniquem seguindo as mesmas regras? O SOAP estabelece padrões que permitem essa interoperabilidade de forma consistente. Derivado de recomendação da W3C, órgão responsável pelos padrões da Web, o protocolo não possui amarras proprietárias.



A comunicação ocorre por meio de mensagens estruturadas em XML, organizadas em um envelope padronizado. Essa estrutura rigorosa permite que consumidores e provedores interpretem os dados corretamente, mesmo utilizando tecnologias completamente diferentes. O que diferencia SOAP de REST? Enquanto REST é um estilo arquitetural, o SOAP define regras específicas para estrutura de mensagens, processamento e tratamento de falhas, o que contribuiu para sua ampla adoção em ambientes empresariais que exigem controle detalhado sobre a comunicação.

Ao longo dos anos, o SOAP tornou-se uma das tecnologias mais utilizadas em Web Services corporativos. Embora muitas aplicações modernas tenham migrado para abordagens mais leves, ele continua presente

em ambientes de missão crítica. A especificação define o SOAP Node, entidade capaz de receber, processar, encaminhar ou gerar mensagens SOAP, e a SOAP Application, a aplicação responsável pela lógica de negócio que produz ou consome essas mensagens, conceitos que ajudam a compreender o fluxo de processamento em arquiteturas SOAP.

Características

Uma das características mais marcantes do SOAP é seu elevado nível de padronização. O protocolo estabelece regras detalhadas para estrutura das mensagens, comunicação entre sistemas e tratamento de falhas. Em ambientes distribuídos, aplicações de fornecedores diferentes precisam compartilhar informações consistentemente: padrões rigorosos reduzem ambiguidades e facilitam a interoperabilidade. Todas as informações transmitidas são organizadas em XML, o que oferece recursos para descrição e validação de dados em estruturas complexas e padronizadas.

O SOAP se destaca também pelo suporte a especificações complementares da família WS-*, que fornecem mecanismos voltados para requisitos corporativos como segurança, confiabilidade, autenticação, roteamento de mensagens e suporte a transações distribuídas. Outra característica relevante é a independência em relação ao protocolo de transporte. O SOAP depende exclusivamente do HTTP? Não. Embora HTTP e HTTPS sejam as opções mais comuns, as mensagens SOAP podem ser transportadas também por SMTP e JMS, ampliando sua aplicabilidade.

Embora a compatibilidade com múltiplos meios de transporte seja uma vantagem teórica do SOAP, o panorama operacional impõe cautelas práticas. A utilização de protocolos como FTP resulta frequentemente em obstruções de pacotes, pois firewalls organizacionais interceptam ativamente essas portas não convencionais. Isso consolida o HTTP e o HTTPS como a opção preferencial hegemônica em ambientes corporativos. Em contrapartida, as mensagens XML tendem a ser mais extensas e exigem maior processamento quando comparadas a formatos mais compactos.

CARACTERÍSTICAS	DESCRIÇÃO
ELEVADO NÍVEL DE PADRONIZAÇÃO	O SOAP define regras rigorosas para a estrutura das mensagens, para a comunicação entre sistemas e para o tratamento de erros. Essa padronização reduz ambiguidades e favorece a interoperabilidade entre aplicações desenvolvidas com tecnologias distintas.
BASEADO EM XML	Todas as mensagens SOAP são representadas em XML. Esse formato permite descrever dados de forma estruturada, padronizada e validável, facilitando a troca de informações entre sistemas heterogêneos.
ESTRUTURA DE MENSAGENS PADRONIZADA	As mensagens SOAP seguem uma estrutura bem definida composta por elementos como Envelope, Header, Body e Fault, garantindo consistência na comunicação entre consumidores e provedores.
INTEROPERABILIDADE	Por utilizar padrões amplamente reconhecidos, o SOAP permite que aplicações desenvolvidas em diferentes linguagens, plataformas e sistemas operacionais consigam trocar informações de maneira transparente.
SUPOORTE À FAMÍLIA WS-*	O SOAP pode ser complementado por especificações da família WS-*, que adicionam funcionalidades relacionadas à segurança, autenticação, autorização, confiabilidade, políticas de comunicação e transações distribuídas.
SEGURANÇA AVANÇADA	Por meio do WS-Security e de outros padrões complementares, o SOAP oferece mecanismos robustos de assinatura digital, criptografia, autenticação baseada em tokens e controle de acesso.
SUPOORTE A TRANSAÇÕES DISTRIBUÍDAS	O ecossistema SOAP permite implementar mecanismos de coordenação e controle de transações que envolvem múltiplos sistemas, recurso muito valorizado em ambientes corporativos.

CARACTERÍSTICAS	DESCRIÇÃO
INDEPENDÊNCIA DE PLATAFORMA	O protocolo não depende de linguagens de programação, sistemas operacionais ou tecnologias específicas, permitindo integração entre ambientes heterogêneos.
INDEPENDÊNCIA DE TRANSPORTE	Embora normalmente utilize HTTP ou HTTPS, o SOAP pode ser transportado por outros protocolos e mecanismos, como SMTP e JMS, oferecendo maior flexibilidade arquitetural.
EXTENSIBILIDADE	A estrutura SOAP permite adicionar novas funcionalidades por meio de cabeçalhos e especificações complementares sem comprometer a compatibilidade com sistemas existentes.
TRATAMENTO PADRONIZADO DE ERROS	O elemento Fault fornece um tratamento estruturado e padronizado de informar erros ocorridos durante o processamento das mensagens.
ORIENTAÇÃO A CONTRATOS	O SOAP normalmente utiliza WSDL para descrever formalmente operações, mensagens, tipos de dados e protocolos, estabelecendo contratos claros entre consumidores e provedores.
MAIOR VERBOSIDADE	Como utiliza XML e estruturas detalhadas, as mensagens SOAP costumam ser maiores e mais complexas do que as utilizadas em APIs REST baseadas em JSON.
MAIOR SOBRECARGA DE PROCESSAMENTO	A interpretação, validação e processamento de documentos XML geralmente exigem mais recursos computacionais quando comparados a formatos mais leves.
FORTE ADOÇÃO CORPORATIVA	Devido ao suporte a requisitos avançados de segurança, governança e confiabilidade, o SOAP continua sendo amplamente utilizado em integrações bancárias, governamentais e empresariais.

Independência

Um dos objetivos mais importantes do SOAP é permitir a interoperabilidade entre aplicações em diferentes plataformas tecnológicas. Em ambientes corporativos, é comum encontrar sistemas desenvolvidos em linguagens distintas e executados em sistemas operacionais variados. Como aplicações tão diferentes conseguem trocar informações de forma confiável? O SOAP estabelece padrões comuns de comunicação: um sistema em Java pode interagir com outro em C# ou Python, pois todos seguem as mesmas regras para estruturação de mensagens e contratos.

O XML desempenha papel importante nesse contexto. Por que esse formato foi amplamente utilizado em arquiteturas SOAP? Como XML é um padrão reconhecido por diferentes plataformas, torna-se possível representar dados de maneira consistente entre sistemas distintos. Essa interoperabilidade oferece vantagens importantes: seria necessário reescrever todos os sistemas existentes para adotar uma única tecnologia? Em muitos casos, não. O SOAP permite que aplicações legadas e soluções mais recentes coexistam, preservando os investimentos realizados.

Padronização

A padronização é uma das características mais importantes do SOAP e desempenhou papel fundamental em sua adoção por grandes organizações. Como garantir que aplicações construídas por fornecedores distintos consigam se comunicar sem conflitos de interpretação? O SOAP foi concebido com base em especificações abertas e amplamente reconhecidas. A estrutura das mensagens, os mecanismos de

tratamento de falhas e os recursos de extensão seguem definições formalmente estabelecidas, tornando a comunicação mais previsível e reduzindo problemas de compatibilidade.

O ecossistema SOAP inclui especificações complementares que ampliam suas capacidades. Como os contratos de serviço são descritos? Como estruturas de dados podem ser validadas? Como mecanismos avançados de segurança são implementados? Tecnologias como WSDL, XML Schema e WS-Security foram desenvolvidas para essas necessidades. Em projetos corporativos, padrões bem definidos permitem que equipes trabalhem com expectativas claramente estabelecidas, reduzindo ambiguidades e aumentando a previsibilidade dos serviços em ambientes que exigem estabilidade.

Uso corporativo

O SOAP conquistou ampla adoção em ambientes corporativos por ser projetado para sistemas críticos. Setores como finanças, telecomunicações e governo eletrônico exigem segurança, confiabilidade e rastreabilidade elevadas. Seu ecossistema inclui padrões que implementam autenticação, assinatura digital, criptografia e coordenação de transações distribuídas. Plataformas corporativas passaram a oferecer recursos nativos para geração automática de contratos e publicação de serviços, reduzindo significativamente o esforço de implementação.

Em ambientes com muitos sistemas, surge a necessidade de uma camada intermediária para coordenar integrações. É nesse contexto que aparece o Enterprise Service Bus, o ESB, que facilita a comunicação por meio de roteamento de mensagens, transformação de dados e mediação entre protocolos. Uma estratégia associada ao ESB é o modelo de dados canônico: as mensagens são convertidas para uma representação padronizada e depois transformadas para o formato exigido pelo sistema de destino, reduzindo o acoplamento em ambientes corporativos complexos.

Mesmo com a popularização de arquiteturas REST e microsserviços, o SOAP continua presente em cenários empresariais. Por que uma tecnologia criada há tantos anos ainda permanece em utilização? A combinação entre maturidade tecnológica, forte padronização e suporte a requisitos corporativos avançados faz com que sistemas bancários, governamentais e de ERP continuem utilizando Web Services SOAP como mecanismo de integração. O protocolo representa uma opção consolidada para comunicação segura e controlada entre plataformas heterogêneas de grande porte.

Estrutura da Mensagem

INCIDÊNCIA EM PROVA: ALTÍSSIMA

A comunicação em Web Services SOAP ocorre por meio de mensagens estruturadas em XML. Para garantir a interoperabilidade, o protocolo estabelece uma estrutura padronizada composta pelos elementos Envelope, Header e Body. Quando ocorre falha, pode ser utilizado o elemento Fault, inserido dentro do Body. O Envelope funciona como elemento raiz que engloba todos os demais. O Header armazena informações complementares ao processamento, enquanto o Body contém os dados associados à operação, preservando a organização e extensibilidade do protocolo.

Além da estrutura básica, diferentes estilos de utilização podem ser adotados em serviços SOAP. O estilo RPC representa chamadas remotas de forma semelhante à invocação de procedimentos locais. Existe outra abordagem? Sim. O estilo Document prioriza a troca de documentos estruturados entre aplicações, concentrando-se nos dados transmitidos. Atualmente, o modelo Document/Literal tornou-se predominante nas implementações corporativas. Para transporte de conteúdos binários, foram desenvolvidas soluções como SOAP with Attachments e MTOM.

Outro recurso relevante é o conceito de `encodingStyle`, presente principalmente em versões mais antigas do protocolo. Qual era sua finalidade? Esse atributo indicava regras de serialização utilizadas para representar tipos de dados dentro da mensagem SOAP. Com a popularização das abordagens baseadas em Document/Literal, sua utilização tornou-se menos frequente. A estrutura padronizada favorece a interoperabilidade, pois consumidores e provedores seguem regras comuns para construir e interpretar mensagens, facilitando a segurança e a auditoria.



Envelope

O Envelope é o elemento raiz de toda mensagem SOAP e constitui a base da estrutura do protocolo para organizar a comunicação. Sempre que uma mensagem SOAP é enviada ou recebida, ela precisa estar contida dentro desse elemento. Por que ele é considerado tão importante? Porque sua presença identifica formalmente a mensagem como compatível com as especificações do SOAP. O Envelope também concentra as declarações de namespaces utilizadas, permitindo que sistemas distingam elementos do protocolo de outros elementos XML presentes na mensagem.

Dentro do Envelope encontram-se os demais elementos do protocolo. O Header armazena informações complementares relacionadas ao processamento, como dados de segurança ou roteamento. O Body contém as informações efetivamente associadas à operação executada. Quando ocorre uma falha, o elemento Fault pode ser incluído dentro do Body para descrever o problema identificado. O que aconteceria sem o Envelope? Os sistemas deixariam de reconhecer a mensagem como compatível com o protocolo, comprometendo sua interpretação e o processamento da operação solicitada.

Header

O Header é um elemento opcional da mensagem SOAP utilizado para transportar informações complementares ao processamento da comunicação. Enquanto o Body contém os dados diretamente associados à operação, o Header armazena metadados auxiliares. Que tipo de informação costuma ser enviada nesse componente? É comum encontrar dados relacionados à autenticação, segurança,

monitoramento, roteamento de mensagens e controle de transações, sem alterar a estrutura principal das operações disponibilizadas pelo serviço ao consumidor.

Uma característica importante do Header é sua capacidade de extensão. Diferentes especificações podem adicionar informações próprias para atender requisitos de um ambiente corporativo. Como implementar recursos avançados de segurança sem modificar as operações da aplicação? Tecnologias como WS-Security utilizam o Header para transportar certificados digitais, tokens de autenticação e assinaturas digitais. Outro conceito relevante é o atributo `mustUnderstand`, que indica que determinados elementos devem ser obrigatoriamente processados pelo destinatário.

Quando um nó SOAP recebe uma mensagem contendo elemento marcado com `mustUnderstand` e não consegue interpretá-lo corretamente, não deve simplesmente ignorá-lo. O processamento resulta na geração de uma SOAP Fault, informando que o requisito obrigatório não pôde ser atendido. Como garantir que informações críticas de segurança não sejam desconsideradas? Ao marcar elementos como obrigatórios, o emissor estabelece que devem ser reconhecidos para que a mensagem seja válida, fortalecendo a consistência e a confiabilidade das integrações baseadas em SOAP.

Body

O Body é o elemento da mensagem SOAP responsável por transportar as informações diretamente relacionadas à operação executada. Sempre que um consumidor solicita uma funcionalidade, os dados necessários são inseridos nesse componente. Quando o serviço processa a solicitação e gera uma resposta, os resultados também são enviados por meio do Body, estruturados conforme o contrato do serviço. Essa padronização garante que ambas as partes interpretem corretamente os dados trocados durante a comunicação entre os sistemas envolvidos.

Uma característica importante do Body é seu caráter obrigatório: toda mensagem SOAP deve conter esse elemento, diferentemente do Header, que pode ou não estar presente. Sem o Body, não existiriam dados de negócio para serem processados. Além de transportar requisições e respostas, o Body também pode comunicar falhas ocorridas durante o processamento. Como o serviço informa que não conseguiu concluir determinada operação? O elemento Fault é inserido dentro do Body, fornecendo informações sobre o erro identificado durante a execução.

Fault

O Fault é o elemento utilizado pelo protocolo SOAP para representar erros ocorridos durante o processamento de uma mensagem. Nem toda solicitação é concluída com sucesso. O que acontece quando uma operação não pode ser executada? O serviço retorna uma resposta contendo um Fault, informando a falha e fornecendo detalhes sobre o problema. É comum a estrutura conter códigos de erro, descrições textuais e dados adicionais que ajudam a identificar a origem, permitindo que aplicações decidam como reagir adequadamente ao incidente.

Uma característica importante do Fault está na padronização. Como aplicações desenvolvidas em tecnologias diferentes conseguem interpretar erros de maneira consistente? O protocolo SOAP define uma estrutura específica para representação das falhas, permitindo que consumidores reconheçam e processem essas informações de forma previsível. O Fault não está presente em todas as mensagens SOAP – ele aparece apenas quando ocorre algum problema durante a execução da operação. Ele é inserido dentro do Body e representa a resposta em situações de erro.

A utilização padronizada do Fault aumenta a confiabilidade da comunicação entre aplicações. A estrutura comum para representação de erros facilita a análise e reduz ambiguidades. Para ilustrar como funciona uma mensagem SOAP, considere uma requisição ao procedimento `retornaNome` com CPF como

parâmetro: o Envelope contém o Header com autenticação e o Body com o dado. Na resposta, o serviço retorna o nome associado ao CPF informado, e o elemento Fault aparece vazio, pois não houve falhas na execução da operação solicitada.

Compreender os quatro elementos – Envelope, Header, Body e Fault – é suficiente para interpretar corretamente a estrutura de uma mensagem SOAP. O Envelope delimita toda a mensagem; o Header adiciona funcionalidades como autenticação; o Body transporta a operação, seus parâmetros e resultados; e o Fault, quando presente no Body, indica que houve um problema durante o processamento. Não é necessário memorizar o código em detalhe: basta saber a função de cada elemento para identificá-los em uma mensagem SOAP real com facilidade.

ELEMENTOS	SITUAÇÃO	DESCRIÇÃO
ENVELOPE (ENVELOPE)	OBRIGATÓRIO	O Envelope é o elemento principal de uma mensagem SOAP e funciona como seu contêiner externo. Ele delimita o início e o fim da mensagem e identifica o documento como uma mensagem SOAP válida. Todos os demais componentes, como Header, Body e Fault, ficam contidos dentro do Envelope. Sua presença é obrigatória em qualquer mensagem SOAP e garante a padronização da estrutura de comunicação.
CABEÇALHO (HEADER)	OPCIONAL	O Header é uma seção opcional da mensagem SOAP destinada ao transporte de informações complementares relacionadas ao processamento da comunicação. Nele podem ser incluídos dados de autenticação, controle de transações, roteamento, segurança e informações necessárias ao tratamento da mensagem. Como é separado do conteúdo principal, permite adicionar funcionalidades sem alterar os dados efetivos da operação solicitada.
CORPO (BODY)	OBRIGATÓRIO	Elemento da mensagem SOAP que contém os dados efetivamente relacionados à operação executada. É nele que são armazenados os parâmetros enviados pelo consumidor e as respostas retornadas pelo provedor. Diferentemente do Header, que possui informações auxiliares, o Body concentra o conteúdo principal da comunicação. Sua presença é obrigatória em qualquer mensagem SOAP válida.
FALHA (FAULT)	OPCIONAL	O Fault é um elemento opcional utilizado para representar erros ocorridos durante o processamento de uma mensagem SOAP. Quando presente, ele fornece informações detalhadas sobre a falha, incluindo códigos de erro, descrições e possíveis causas do problema. Esse mecanismo padroniza o tratamento de exceções, permitindo que consumidores interpretem e respondam adequadamente aos erros ocorridos durante a comunicação.

WSDL

INCIDÊNCIA EM PROVA: ALTÍSSIMA

WSDL

WSDL (Web Services Description Language) é uma linguagem baseada em XML utilizada para descrever formalmente um Web Service. Por meio dela, são especificadas operações disponíveis, mensagens suportadas, formatos de dados utilizados, protocolos de comunicação e endereços de acesso. O WSDL funciona como um contrato técnico detalhado que permite a integração automática entre consumidores e provedores.

WSDL, Web Services Description Language, é uma linguagem baseada em XML utilizada para descrever formalmente um Web Service. Quando um sistema deseja consumir um serviço, precisa compreender quais funcionalidades estão disponíveis e como acessá-las. O WSDL foi criado para fornecer essa descrição técnica detalhada. No ecossistema SOAP, ele desempenha o papel de contrato formal, especificando operações disponibilizadas, mensagens utilizadas, tipos de dados, protocolos de comunicação e pontos de acesso, reduzindo ambiguidades nas integrações.

Uma das características mais valorizadas do WSDL está relacionada à automação do desenvolvimento. Seria necessário implementar manualmente toda a estrutura de comunicação para consumir um serviço SOAP? Em muitos casos, não. Diversas ferramentas conseguem interpretar o documento WSDL e gerar automaticamente classes, métodos e estruturas de dados compatíveis com o serviço. Essa capacidade reduz o esforço de implementação e diminui a probabilidade de erros na construção dos clientes consumidores em diferentes plataformas tecnológicas.

O WSDL também desempenha papel importante na manutenção e evolução dos serviços. Quando uma nova aplicação precisa ser integrada ao ambiente, não é necessário analisar a implementação interna do provedor para compreender como a comunicação deve ocorrer. Basta consultar o contrato disponibilizado. Essa separação entre descrição e implementação favorece o desacoplamento entre sistemas e facilita a expansão de arquiteturas baseadas em Web Services SOAP, permitindo que equipes diferentes desenvolvam soluções compatíveis de forma independente.

WSDL: O GUIA VISUAL DO WEB SERVICES DESCRIPTION LANGUAGE

SKETCHNOTE DIDÁTICO & PEDAGÓGICO



Contrato do Serviço

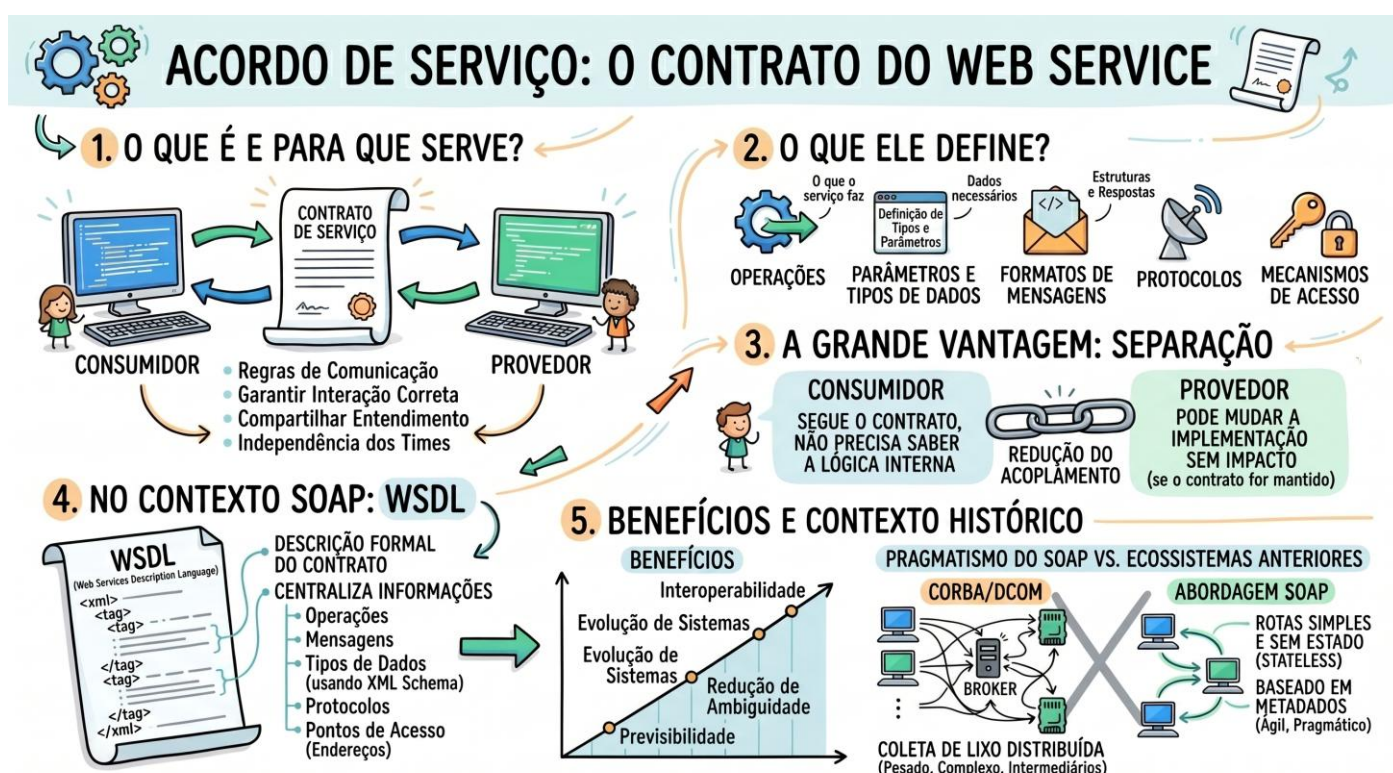
CONTRATO DE SERVIÇO

O contrato do serviço é a especificação que define como um serviço deve ser utilizado por seus consumidores. Ele descreve operações, parâmetros, tipos de dados, formatos de mensagens, protocolos e demais regras de comunicação. Seu objetivo é garantir que diferentes aplicações possam interagir corretamente sem necessidade de conhecer detalhes internos da implementação. Em ambientes SOAP, o WSDL costuma representar esse contrato.

O contrato de serviço define como consumidores e provedores devem se comunicar em Web Services. Como garantir que sistemas desenvolvidos por equipes diferentes consigam interagir corretamente? O contrato estabelece formalmente as condições necessárias, especificando operações disponíveis, dados exigidos em cada requisição, respostas possíveis e estruturas utilizadas para representar informações. Além disso, podem ser definidos protocolos de comunicação e mecanismos de acesso para que o serviço seja consumido de maneira adequada e previsível.

A principal vantagem dessa abordagem está na separação entre interface e implementação. O consumidor precisa conhecer a lógica interna do sistema provedor? Não – basta compreender as regras do contrato. No contexto do SOAP, a descrição formal é realizada por meio do WSDL, que reúne operações, mensagens, tipos de dados baseados em XML Schema, protocolos e pontos de acesso do serviço. Todas as informações necessárias para consumir o serviço permanecem centralizadas nessa única especificação padronizada e amplamente reconhecida.

A existência de contratos bem definidos contribui para a interoperabilidade dos sistemas ao longo do tempo. Como equipes diferentes desenvolvem aplicações compatíveis sem contato constante? Uma descrição formal compartilhada reduz ambiguidades durante o desenvolvimento. Parte do sucesso inicial do SOAP residiu no descolamento de arquiteturas como CORBA e DCOM, que requisitavam componentes intermediários custosos. O SOAP preferiu rotas apátridas baseadas em metadados, tornando a integração mais ágil e dispensando o monitoramento complexo de instâncias.



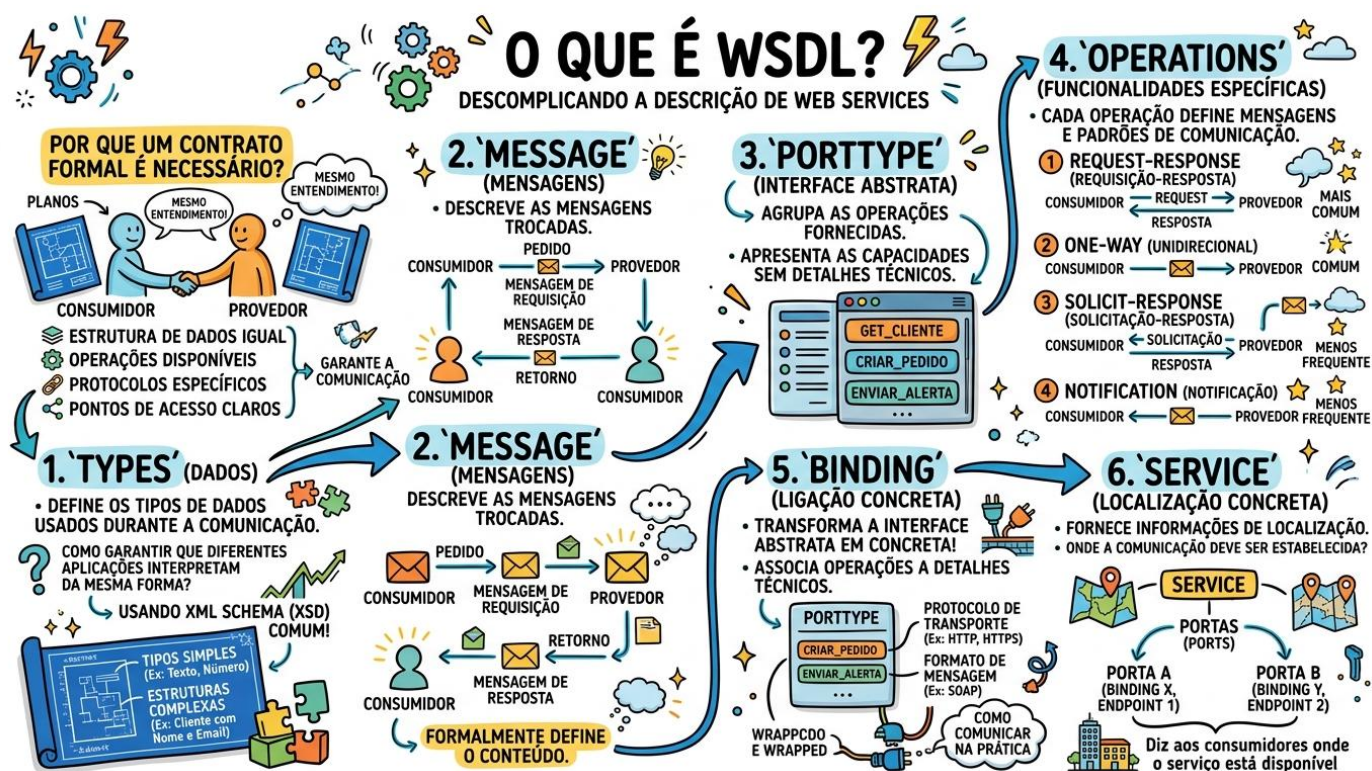
Estruturas principais

A descrição de um Web Service em WSDL é composta por elementos com responsabilidades específicas. O elemento Types define os tipos de dados utilizados na comunicação, normalmente com base em XML Schema, desde tipos simples até estruturas complexas. O elemento Message descreve as mensagens trocadas entre consumidor e provedor, definindo quais informações compõem requisições e respostas. A interface abstrata do serviço é representada pelo PortType, que agrupa as operações disponibilizadas sem detalhar protocolos ou mecanismos concretos de acesso.

Dentro do PortType encontram-se as Operations, que representam funcionalidades específicas e definem quais mensagens participam da comunicação. O WSDL 1.1 apresenta quatro modelos: Request-Response, em que o consumidor envia requisição e recebe resposta; One-Way, com apenas envio sem resposta; Solicit-Response, em que o serviço inicia a interação; e Notification, em que o serviço envia

notificações sem expectativa de retorno. Os dois últimos são pouco frequentes em implementações modernas, mas fazem parte da especificação.

Após definir operações e mensagens, torna-se necessário especificar como a comunicação ocorrerá na prática. Essa função é desempenhada pelo elemento Binding, que associa as operações do PortType a detalhes técnicos como protocolo de transporte e formato das mensagens. Em implementações SOAP, é comum especificar SOAP sobre HTTP ou HTTPS. Por fim, o elemento Service informa onde o serviço está disponível na rede, reunindo ports com bindings e endereços concretos de acesso, completando a estrutura de descrição definida pelo WSDL.



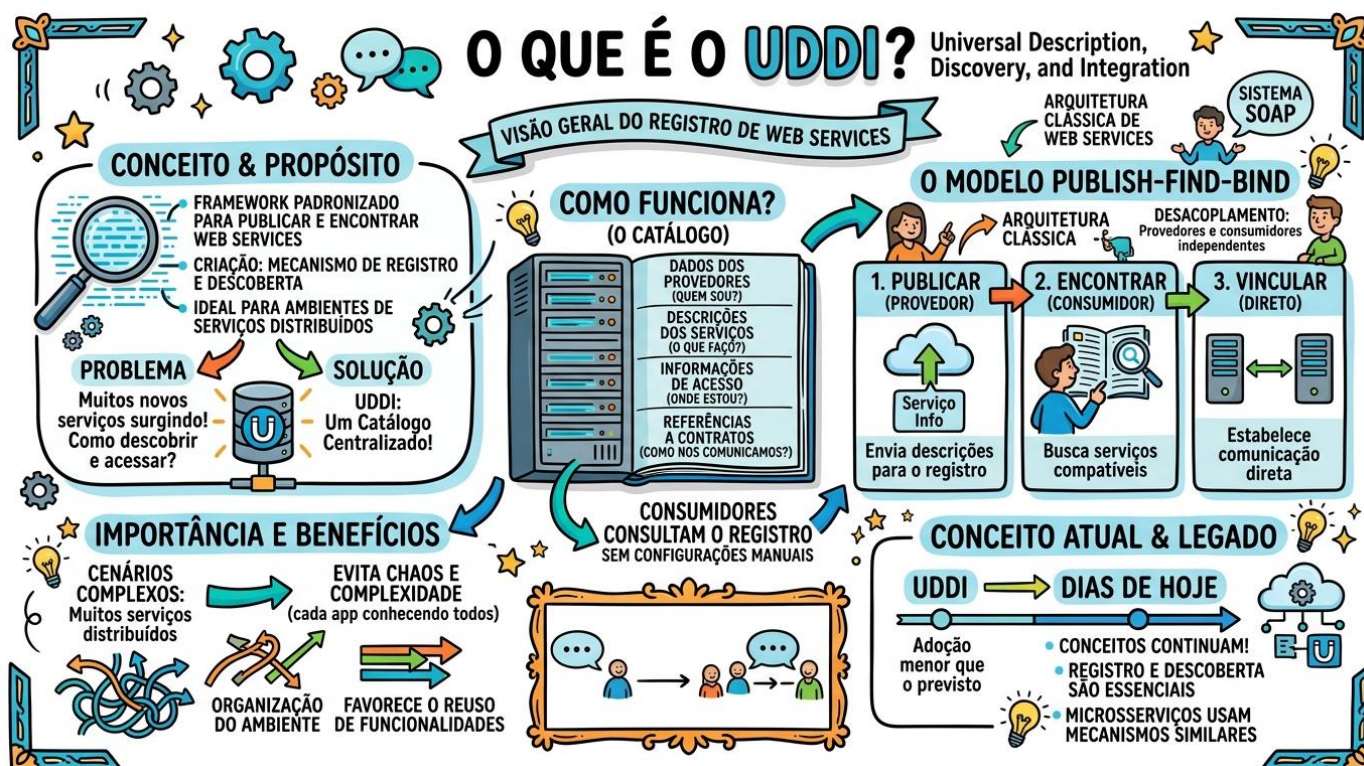
UDDI

INCIDÊNCIA EM PROVA: MÉDIA

UDDI

UDDI (Universal Description, Discovery and Integration) é uma especificação criada para permitir o registro e a localização de Web Services. Ela fornece uma estrutura padronizada para que organizações publiquem informações sobre seus serviços e para que consumidores possam encontrá-los. Embora tenha perdido relevância prática ao longo do tempo, o UDDI continua sendo importante para compreender os modelos clássicos de SOA.

O UDDI, Universal Description, Discovery and Integration, foi criado para atuar como mecanismo de registro e descoberta de serviços em Web Services. Em arquiteturas distribuídas, novos serviços surgem continuamente disponibilizados por diferentes organizações. Como uma aplicação descobre quais serviços existem e como acessá-los? O UDDI fornece um catálogo padronizado capaz de armazenar informações sobre serviços, incluindo dados sobre provedores, descrições, informações de acesso e referências aos contratos utilizados na comunicação.



O UDDI desempenhava papel importante na arquitetura clássica dos Web Services. Em vez de estabelecer conexões previamente configuradas entre todas as aplicações, os serviços podiam ser publicados em um registro central, e os consumidores consultavam esse registro para localizá-los e estabelecer comunicação com o provedor. Essa abordagem está associada ao modelo Publish-Find-Bind. A proposta tornava-se especialmente relevante em cenários com grande quantidade de serviços distribuídos, onde a administração manual seria extremamente complexa.

Embora o UDDI tenha sido considerado um componente estratégico durante o período de maior expansão dos Web Services SOAP, não alcançou o nível de adoção inicialmente previsto. Isso significa que seus conceitos perderam relevância? Não. As ideias de registro, localização e descoberta de serviços continuam presentes em diversas arquiteturas modernas. Muitos ambientes baseados em microsserviços utilizam mecanismos que desempenham funções semelhantes, demonstrando que os princípios introduzidos pelo UDDI continuam influenciando a integração de sistemas.

Registro de Serviços

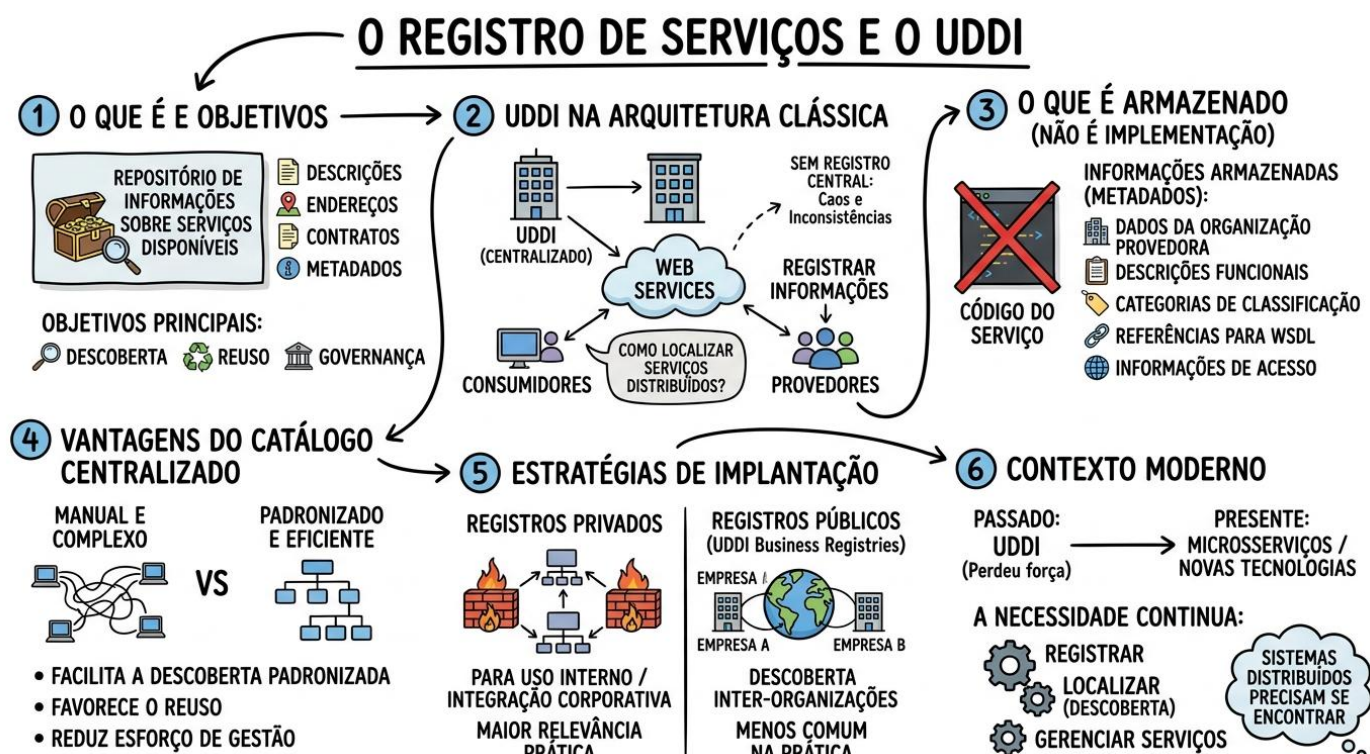
REGISTRO DE SERVIÇOS

O registro de serviços é um repositório utilizado para armazenar informações sobre serviços disponíveis em determinado ambiente. Ele contém descrições, endereços de acesso, contratos e metadados que facilitam a localização e utilização dos serviços. Seu principal objetivo é promover descoberta, reutilização e governança dos recursos disponibilizados por diferentes provedores dentro de uma arquitetura orientada a serviços.

O principal papel do UDDI era atuar como registro de serviços dentro da arquitetura clássica dos Web Services. Como localizar serviços disponíveis sem depender de configurações manuais para cada integração? O UDDI funcionava como catálogo centralizado armazenando dados sobre a organização provedora, descrições funcionais, categorias de classificação, referências a documentos WSDL e informações de acesso. Esses metadados permitiam que consumidores localizassem serviços adequados e compreendessem como realizar a comunicação com o provedor.

A existência de um catálogo centralizado oferecia vantagens importantes. O que aconteceria se cada aplicação precisasse manter manualmente informações sobre todos os serviços existentes? O gerenciamento das integrações se tornaria difícil e sujeito a inconsistências. A especificação UDDI também permitia diferentes estratégias de implantação: registros privados, acessíveis apenas internamente, e registros públicos, conhecidos como UDDI Business Registries. Na prática, os registros privados tiveram maior relevância do que os modelos públicos idealizados.

Embora o UDDI tenha perdido espaço ao longo dos anos, os princípios que motivaram sua criação continuam presentes em arquiteturas modernas. Como sistemas distribuídos localizam serviços em ambientes compostos por centenas de aplicações? Atualmente, diversas plataformas utilizam mecanismos próprios de catálogo e descoberta de serviços. A tecnologia mudou, mas a necessidade de registrar, localizar e gerenciar serviços continua sendo um requisito importante em arquiteturas distribuídas contemporâneas, demonstrando a influência duradoura do UDDI.



Publicação

PUBLICAÇÃO

Publicação é o processo pelo qual um provedor registra informações sobre um serviço em um mecanismo de registro. Durante essa etapa, são disponibilizados dados como descrição, endereço de acesso, contrato e características do serviço. A publicação torna o serviço visível para potenciais consumidores, permitindo que ele seja localizado e utilizado por outras aplicações dentro do ambiente de integração.

A publicação representa a etapa em que um provedor registra informações sobre seus serviços em um repositório UDDI. Não basta disponibilizar uma funcionalidade na rede – os potenciais consumidores precisam saber que ela existe. Para isso, o provedor realiza a publicação, tornando as informações acessíveis para consulta. São cadastrados dados sobre a organização responsável, descrições funcionais, categorias de classificação, informações de contato, endereços de acesso e referências aos documentos WSDL que descrevem formalmente os contratos dos serviços.

A principal finalidade da publicação é tornar os serviços visíveis dentro do catálogo de integração, correspondendo ao primeiro componente do modelo Publish-Find-Bind. Após o registro ser concluído, o serviço passa a integrar o conjunto de recursos disponíveis para descoberta pelos consumidores, que podem localizar funcionalidades existentes sem depender de conhecimento prévio sobre os sistemas. A proposta original do UDDI buscava criar ecossistemas dinâmicos, embora a adoção prática tenha sido mais limitada do que as expectativas iniciais.

A etapa de publicação também contribui para a organização e governança dos serviços disponíveis. Como administrar um ambiente composto por dezenas ou centenas de integrações sem um catálogo centralizado? O registro das informações em um repositório estruturado facilita o controle dos recursos existentes e fornece uma visão consolidada das funcionalidades disponíveis. Essa necessidade de catalogar e gerenciar serviços continua presente em arquiteturas distribuídas contemporâneas, mesmo quando tecnologias diferentes do UDDI são utilizadas nesse papel.



Descoberta

DESCOBERTA

Descoberta é o processo de localizar serviços disponíveis em um ambiente distribuído. Normalmente, um consumidor consulta um registro de serviços em busca de funcionalidades que atendam às suas necessidades. A descoberta reduz dependências diretas entre aplicações e facilita a reutilização de recursos existentes, permitindo que novos consumidores encontrem serviços sem necessidade de conhecimento prévio sobre sua localização.

A descoberta corresponde à etapa em que consumidores consultam um registro UDDI para localizar serviços capazes de atender suas necessidades. Em ambientes distribuídos, como encontrar um serviço adequado sem conhecer previamente sua localização? O mecanismo permite pesquisar um catálogo centralizado em busca de funcionalidades específicas utilizando critérios como nomes, categorias, organizações responsáveis ou descrições funcionais. Após identificar um serviço compatível, tornam-se disponíveis referências ao WSDL e aos pontos de acesso do provedor.



A proposta da descoberta dinâmica representava um dos pilares da visão original dos Web Services. Por que essa ideia foi considerada tão importante? Porque permitia reduzir a dependência de configurações previamente estabelecidas entre consumidores e provedores. As aplicações poderiam consultar o registro para localizar serviços sempre que necessário, favorecendo o desacoplamento entre os participantes. Na prática, a adoção desse modelo ocorreu de forma mais limitada do que o previsto, porém os conceitos continuam presentes em arquiteturas modernas.

A etapa de descoberta contribui para a reutilização de funcionalidades já existentes. O que acontece quando uma nova aplicação precisa executar determinada tarefa? Em vez de desenvolver uma solução completamente nova, ela pode localizar serviços previamente disponibilizados. Essa capacidade de identificar e reutilizar recursos é um dos fatores que motivaram a criação dos registros de serviços e continua influenciando o projeto de arquiteturas distribuídas contemporâneas. A descoberta completa o ciclo do modelo Publish-Find-Bind dos Web Services.

Web Services REST

Conceitos Básicos

INCIDÊNCIA EM PROVA: ALTÍSSIMA

REST

Web Services REST são serviços construídos segundo os princípios da arquitetura REST, utilizando recursos identificados por URIs e operações baseadas nos métodos HTTP. Diferentemente do SOAP, REST não é um protocolo, mas um estilo arquitetural. Sua simplicidade, flexibilidade e eficiência contribuíram para sua ampla adoção em APIs modernas, especialmente em aplicações Web, móveis e ambientes de microsserviços.

REST, Representational State Transfer, é um estilo arquitetural criado para orientar o desenvolvimento de sistemas distribuídos que utilizam a infraestrutura da Web. Muitas pessoas associam REST a um protocolo específico. Essa associação está correta? Não. REST não define mensagens, formatos ou protocolos próprios. Sua proposta consiste em estabelecer princípios arquiteturais que auxiliam na construção de serviços simples, escaláveis e interoperáveis, aproveitando recursos já existentes na própria arquitetura da Web, como URIs e métodos HTTP.

O foco principal do REST está nos recursos disponibilizados pelo sistema. O que é um recurso? Trata-se de qualquer informação que possa ser identificada por meio de uma URL, como clientes, produtos ou pedidos. Os consumidores não manipulam diretamente esses recursos, mas suas representações. Quando uma requisição é realizada, o servidor retorna uma representação do estado atual do recurso, normalmente em JSON ou XML. O JSON tornou-se predominante nas APIs REST por sua estrutura simples e geração de mensagens mais compactas.

A adoção dos princípios REST transformou significativamente a forma de construir APIs para integração entre sistemas. Como conectar aplicações web, dispositivos móveis, plataformas corporativas e serviços em nuvem de maneira eficiente? A resposta encontrada pela indústria foi utilizar uma arquitetura alinhada aos fundamentos da própria Web. REST não impõe um formato específico de dados, permitindo diferentes representações conforme os requisitos da aplicação, o que favorece sua ampla adoção em sistemas distribuídos modernos.

Estilo Arquitetural

ESTILO ARQUITETURAL

Um estilo arquitetural é um conjunto de princípios, restrições e diretrizes que orientam a construção de sistemas de software. Em vez de definir tecnologias específicas, estabelece padrões de organização e interação entre componentes. REST é um exemplo de estilo arquitetural, pois descreve características que devem ser seguidas para que um sistema possa ser considerado aderente aos seus princípios.

REST é classificado como um estilo arquitetural utilizado no projeto de sistemas distribuídos. O que isso significa? REST não é um protocolo, uma linguagem de programação ou uma tecnologia específica. Em vez disso, fornece princípios que orientam a comunicação entre componentes de um sistema. Aplicações podem ser desenvolvidas com diferentes linguagens e plataformas, desde que respeitem as restrições estabelecidas. Um dos pilares é o conceito de recurso: clientes interagem com recursos por URIs sem precisar conhecer a implementação interna do provedor.

A separação entre cliente e servidor ocupa papel central na arquitetura REST. Por que essa divisão é importante? Porque cada componente possui responsabilidades bem definidas: o cliente concentra a interface e a interação com o usuário; o servidor mantém os dados e executa as regras de negócio. REST

também incorpora outras restrições arquiteturais: comunicação stateless, interface uniforme, possibilidade de cache e organização em camadas. Essas restrições favorecem escalabilidade, interoperabilidade, simplicidade e desempenho em sistemas distribuídos.

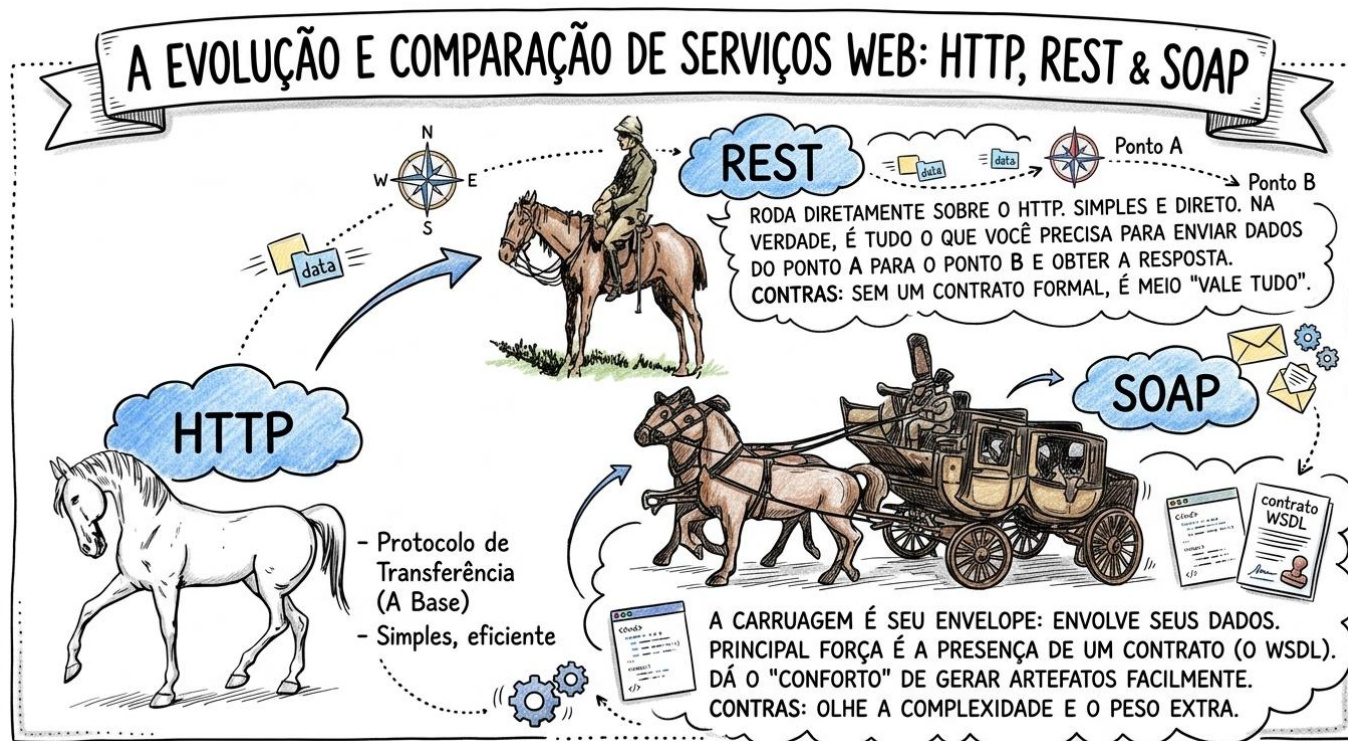
REST x SOAP

REST X SOAP

REST e SOAP representam abordagens distintas para implementação de Web Services. SOAP é um protocolo formal baseado em XML, com forte padronização e amplo suporte a requisitos corporativos avançados. REST é um estilo arquitetural mais simples e flexível, geralmente associado ao uso de HTTP e JSON. Enquanto SOAP prioriza padronização rigorosa, REST enfatiza simplicidade, desempenho e facilidade de integração.

REST e SOAP são abordagens para comunicação entre sistemas distribuídos, mas com características distintas. SOAP é um protocolo formal com regras específicas para troca de mensagens, enquanto REST é um estilo arquitetural que descreve princípios de projeto. As diferenças também aparecem na representação dos dados: SOAP define mensagens obrigatoriamente em XML, organizadas em Envelope, Header e Body. REST não exige formato específico, embora o JSON seja amplamente utilizado por sua estrutura simples e compacta em aplicações modernas.

Outro aspecto frequentemente discutido envolve a complexidade das implementações. O SOAP oferece mais recursos? Em muitos cenários corporativos, sim. O ecossistema SOAP é complementado pela família WS-*, fornecendo mecanismos de segurança, confiabilidade e transações distribuídas. REST adota abordagem diferente, aproveitando recursos consolidados da Web. Diante dessas diferenças, qual abordagem escolher? A resposta depende dos requisitos: APIs para aplicações web tendem a adotar REST, enquanto ambientes que exigem contratos rigorosos utilizam SOAP.



Principais Restrições

INCIDÊNCIA EM PROVA: ALTA

REST foi definido por Roy Fielding como um estilo arquitetural para sistemas distribuídos. REST é apenas uma forma de utilizar HTTP? A resposta é negativa. O foco do REST está em um conjunto de restrições arquiteturais que orienta a comunicação entre clientes e servidores. Essas restrições foram formuladas a partir da análise das propriedades que tornaram a Web eficiente e escalável, buscando favorecer interoperabilidade, desempenho, simplicidade e capacidade de evolução dos sistemas ao longo do tempo com previsibilidade.

As restrições propostas são: Client-Server, Stateless, Cache, Interface Uniforme, Sistema em Camadas e Code on Demand. Esta última possui uma característica particular – é considerada opcional na definição original. As demais compõem os elementos essenciais de uma arquitetura aderente ao REST. Utilizar HTTP e trocar dados em JSON, por si só, não garante conformidade com REST. Uma arquitetura RESTful deve respeitar as restrições definidas, especialmente a ausência de estado nas requisições e a padronização das interações entre componentes.

Client-Server

CLIENT-SERVER

A restrição Cliente-Servidor estabelece que a interface do usuário e o armazenamento ou processamento de dados devem permanecer separados. O cliente é responsável por solicitar recursos, enquanto o servidor fornece serviços e gerencia informações. Essa separação favorece evolução independente dos componentes, melhora a escalabilidade e reduz dependências entre diferentes partes da aplicação.

A restrição Client-Server define separação clara de responsabilidades entre os participantes da comunicação. As responsabilidades relacionadas à interface e à experiência do usuário ficam concentradas no cliente, enquanto o armazenamento dos dados e as regras de negócio permanecem no servidor. Uma consequência importante é a possibilidade de reutilizar os mesmos recursos em diferentes aplicações: um navegador web, um aplicativo móvel, um sistema corporativo ou dispositivo IoT podem acessar os mesmos dados fornecidos pelo servidor de formas distintas.

Essa separação influencia diretamente a manutenção dos sistemas. Alterações em uma aplicação web nem sempre exigem mudanças no servidor porque a interface visual pertence ao cliente. Da mesma forma, o servidor pode receber otimizações internas sem impactar consumidores da API, desde que os contratos de comunicação permaneçam compatíveis. Ao observar essa restrição, fica mais fácil compreender por que ela contribui para o baixo acoplamento, tornando os sistemas mais organizados, escaláveis e preparados para incorporar novos clientes ao longo do tempo.

Stateless

STATELESS

A restrição Stateless determina que cada requisição enviada ao servidor deve conter todas as informações necessárias para seu processamento. O servidor não deve armazenar contexto ou estado relacionado às interações anteriores do cliente. Essa característica simplifica o gerenciamento das aplicações, melhora a escalabilidade e facilita a distribuição de requisições entre diferentes servidores.

A restrição Stateless estabelece que cada requisição deve ser autossuficiente para o seu processamento. O que significa ser autossuficiente? Todas as informações necessárias para executar a operação precisam acompanhar a própria solicitação. O servidor não deve depender de informações sobre interações anteriores com aquele cliente. Isso costuma gerar uma dúvida: o servidor não pode armazenar dados? Pode. A restrição não impede o uso de bancos de dados – ela se refere apenas ao estado da interação entre cliente e servidor.

A ausência de estado compartilhado entre requisições traz benefícios importantes. Por que essa característica favorece a escalabilidade? Como qualquer servidor pode processar qualquer requisição de forma independente, torna-se mais simples distribuir a carga entre múltiplas instâncias. Em implementações RESTful, informações de autenticação acompanham cada chamada, como tokens de acesso. Essa abordagem aumenta o volume de dados transmitidos, mas contribui significativamente para a simplicidade, a confiabilidade e a escalabilidade da arquitetura.

Cache

CACHE

A restrição Cache estabelece que as respostas fornecidas pelos servidores devem indicar se podem ou não ser armazenadas temporariamente pelos clientes ou intermediários. O uso de cache reduz a necessidade de novas requisições para recursos frequentemente acessados, diminuindo o consumo de banda e melhorando o desempenho geral das aplicações.

A restrição Cache foi criada para reduzir a necessidade de consultas repetidas aos servidores. A arquitetura permite que determinadas respostas sejam armazenadas temporariamente e reutilizadas em solicitações futuras. Para que isso ocorra de forma segura, o servidor deve informar explicitamente se uma resposta pode ou não ser armazenada. Quando um recurso é considerado cacheável, o cliente pode utilizar uma cópia armazenada anteriormente em vez de consultar novamente o servidor, reduzindo o tempo de resposta percebido pelo usuário.

Quem decide quando uma informação pode ser reutilizada? Essa responsabilidade pertence ao servidor. Em implementações baseadas em HTTP, existem mecanismos padronizados para definir regras de cache, incluindo tempo de validade e critérios para atualização dos dados. Além dos ganhos de desempenho, o cache exerce papel importante na escalabilidade dos sistemas distribuídos. Quanto menor a quantidade de requisições encaminhadas ao servidor, menor a carga de processamento necessária, permitindo que a mesma infraestrutura suporte maior volume de acessos.

Interface Uniforme

INTERFACE UNIFORME

A Interface Uniforme é considerada a principal restrição da arquitetura REST. Ela define que os recursos devem ser identificados de maneira consistente, acessados por meio de métodos padronizados e representados utilizando formatos adequados. Essa uniformidade simplifica a comunicação entre sistemas e reduz a complexidade das integrações, independentemente das tecnologias utilizadas.

A Interface Uniforme é considerada a restrição central da arquitetura REST. Por que ela recebe tanta atenção? Porque define a forma padronizada pela qual clientes e servidores se comunicam, reduzindo a complexidade das integrações. Um dos pilares é a identificação de recursos: cada um deve possuir identificador único, normalmente uma URI. As operações são realizadas com métodos HTTP como GET, POST, PUT e DELETE, tornando a comunicação mais previsível. Os clientes interagem com representações dos recursos, não com suas implementações internas.

As mensagens devem ser autodescritivas, contendo informações suficientes para serem compreendidas adequadamente. Como o cliente descobre quais ações podem ser executadas a seguir? Na definição original do REST, essa navegação ocorre por meio de hipermídia, princípio conhecido como HATEOAS, no qual as próprias respostas fornecem links e caminhos para novas interações. Esses elementos – identificação de recursos, representações, mensagens autodescritivas e HATEOAS – formam os quatro componentes da Interface Uniforme no estilo REST.

Sistema em Camadas

SISTEMA EM CAMADAS

A restrição Sistema em Camadas determina que a arquitetura pode ser composta por múltiplos níveis intermediários entre cliente e servidor. Esses intermediários podem desempenhar funções como balanceamento de carga, cache, autenticação, monitoramento e segurança. O cliente não precisa conhecer a estrutura interna dessas camadas, interagindo apenas com a interface exposta pelo sistema.

A restrição Sistema em Camadas permite que a comunicação entre cliente e servidor seja organizada por meio de componentes intermediários. Em vez de concentrar todas as responsabilidades em um único componente, a arquitetura pode distribuí-las entre várias camadas. Cada elemento interage apenas com os componentes adjacentes. O cliente sabe exatamente com qual servidor está se comunicando? Nem sempre. A requisição pode atravessar proxies, gateways de API, balanceadores de carga e mecanismos de cache, todos atuando de forma transparente para quem solicita.

A utilização de camadas permite distribuir funções específicas ao longo da infraestrutura. Determinados componentes podem assumir responsabilidades de autenticação, outros de roteamento ou monitoramento de tráfego. Essa especialização reduz a complexidade de cada elemento e torna a arquitetura mais organizada. Novas camadas podem ser adicionadas para atender requisitos específicos sem exigir alterações nos clientes ou servidores existentes. Desde que a interface permaneça compatível, a infraestrutura pode evoluir gradualmente com novos mecanismos.

Code on Demand

CODE ON DEMAND

Code on Demand é uma restrição opcional da arquitetura REST que permite ao servidor enviar código executável para o cliente quando necessário. Esse código pode ampliar funcionalidades temporariamente sem exigir instalação prévia de novos componentes. Embora seja pouco utilizada em APIs modernas, essa característica pode ser observada em aplicações Web que enviam scripts para execução nos navegadores.

Code on Demand ocupa posição particular entre as restrições da arquitetura REST. O que a torna diferente das demais? Trata-se da única restrição considerada opcional na definição original proposta por Roy Fielding. Sua finalidade é permitir que o servidor envie código executável ao cliente, ampliando temporariamente suas capacidades. Novas funcionalidades podem ser incorporadas dinamicamente durante a execução, sem que todas precisem estar previamente implementadas no ambiente cliente, representando grande flexibilidade para o sistema.

Quando um navegador acessa uma página e recebe scripts para execução local, o servidor transmite não apenas dados, mas também código capaz de realizar validações e interações na interface. Uma dúvida frequente: uma API que não utiliza Code on Demand deixa de ser RESTful? Não. Como essa restrição é opcional, sua ausência não compromete a aderência ao estilo arquitetural. Na prática, a utilização de Code on Demand é menos comum, pois grande parte das APIs tem como objetivo principal fornecer recursos e dados para diferentes consumidores.

Recursos e URIs

INCIDÊNCIA EM PROVA: BAIXA

RECURSOS/URI

Um recurso é qualquer entidade ou informação disponibilizada por um sistema que possa ser identificada, acessada e manipulada por meio de uma API. Um recurso pode representar objetos do mundo real, como clientes, produtos, pedidos ou usuários, bem como informações ou serviços oferecidos pela aplicação. Já uma

URI (Uniform Resource Identifier) é o identificador utilizado para localizar e distinguir um recurso dentro de uma API REST. Geralmente representada por um endereço semelhante a uma URL, a URI indica onde determinado recurso pode ser acessado

O conceito de recurso está no centro da arquitetura REST. Ao projetar uma API seguindo esse estilo, a atenção é direcionada aos elementos do domínio de negócio que precisam ser disponibilizados: clientes, produtos, pedidos ou qualquer informação que possa ser identificada e manipulada pela rede. Para que um recurso possa ser acessado, ele precisa de identificador único representado por uma URI. A URI deve representar o recurso, não uma ação – endereços como `"/clientes"` são preferíveis a `"/consultarCliente"` ou `"/removerProduto"`.

As operações são indicadas pelos métodos HTTP. Uma chamada GET para `"/clientes"` solicita a consulta da coleção de clientes. Uma requisição POST para essa mesma URI indica a criação de um novo registro. Já `"/clientes/10"` identifica um recurso individual: uma requisição GET retorna seus dados; PUT atualiza suas informações; DELETE solicita sua remoção. Essa combinação entre URIs representando recursos e métodos HTTP representando operações torna a comunicação previsível e padronizada para todos os consumidores da API.

Esse padrão facilita a compreensão da estrutura da API. Ao observar uma URI, desenvolvedores identificam rapidamente se ela representa uma coleção ou um recurso específico. A consistência na organização dos endereços reduz ambiguidades e simplifica a integração entre sistemas. A leitura da URI fornece pistas importantes sobre o papel do recurso dentro da aplicação e a forma adequada de interagir com ele. Essa clareza é um dos fatores que tornaram a arquitetura REST tão popular em ambientes distribuídos e em APIs modernas.

As URIs também ajudam a representar relacionamentos entre recursos distintos. Em um sistema que gerencia clientes e pedidos, a URI `"/clientes/10"` torna explícito o recurso referenciado. Cada recurso mantém sua própria identidade enquanto permanece conectado aos demais elementos do domínio de negócio. A organização em torno de recursos, combinada com os métodos HTTP para indicar operações, produz interfaces consistentes e previsíveis, favorecendo a interoperabilidade e permitindo que sistemas consumam os recursos sem necessidade de documentação extensa.

CRUD x HTTP

INCIDÊNCIA EM PROVA: MÉDIA

CRUD representa as operações fundamentais de manipulação de dados: Create, Read, Update e Delete, correspondendo a criar, consultar, atualizar e remover. Em APIs REST, existe associação amplamente adotada entre essas operações e os métodos HTTP, permitindo manipulação padronizada dos recursos. A operação Create é realizada via POST enviado para a URI da coleção correspondente. Após processar a solicitação, o servidor costuma retornar o código HTTP 201, indicando que o recurso foi criado com sucesso e passou a existir na aplicação.

A operação Read utiliza o método GET: enviado para `"/clientes"` retorna a coleção; enviado para `"/clientes/10"` retorna um recurso específico. O método GET deve ser utilizado apenas para consultas, sem provocar alterações no estado das informações. Para modificar dados existentes, entram em cena as operações de atualização. O método PUT é associado à substituição completa da representação de um recurso, enquanto o PATCH é empregado para alterações parciais em atributos específicos, expressando a intenção da operação de forma mais clara.

A remoção de recursos corresponde à operação Delete e utiliza o método HTTP DELETE. Quando enviado para uma URI específica, o servidor avalia a requisição e, se permitida, realiza a exclusão. Em muitas implementações, a resposta retorna o código HTTP 204, indicando que a operação foi concluída

sem conteúdo adicional a enviar ao cliente. A associação entre CRUD e HTTP contribui para interfaces consistentes: em vez de comandos específicos para cada funcionalidade, a API utiliza métodos padronizados, facilitando o entendimento e a integração entre sistemas.

OPERAÇÃO CRUD	MÉTODO HTTP	FINALIDADE	EXEMPLO DE ENDPOINT
CREATE	POST	Criar um novo recurso no servidor.	POST /clientes
READ	GET	Consultar ou recuperar um ou mais recursos existentes.	GET /clientes/10
UPDATE	PUT	Atualizar completamente um recurso existente.	PUT /clientes/10
UPDATE	PATCH	Atualizar parcialmente um recurso existente.	PATCH /clientes/10
DELETE	DELETE	Remover um recurso existente.	DELETE /clientes/10

JSON

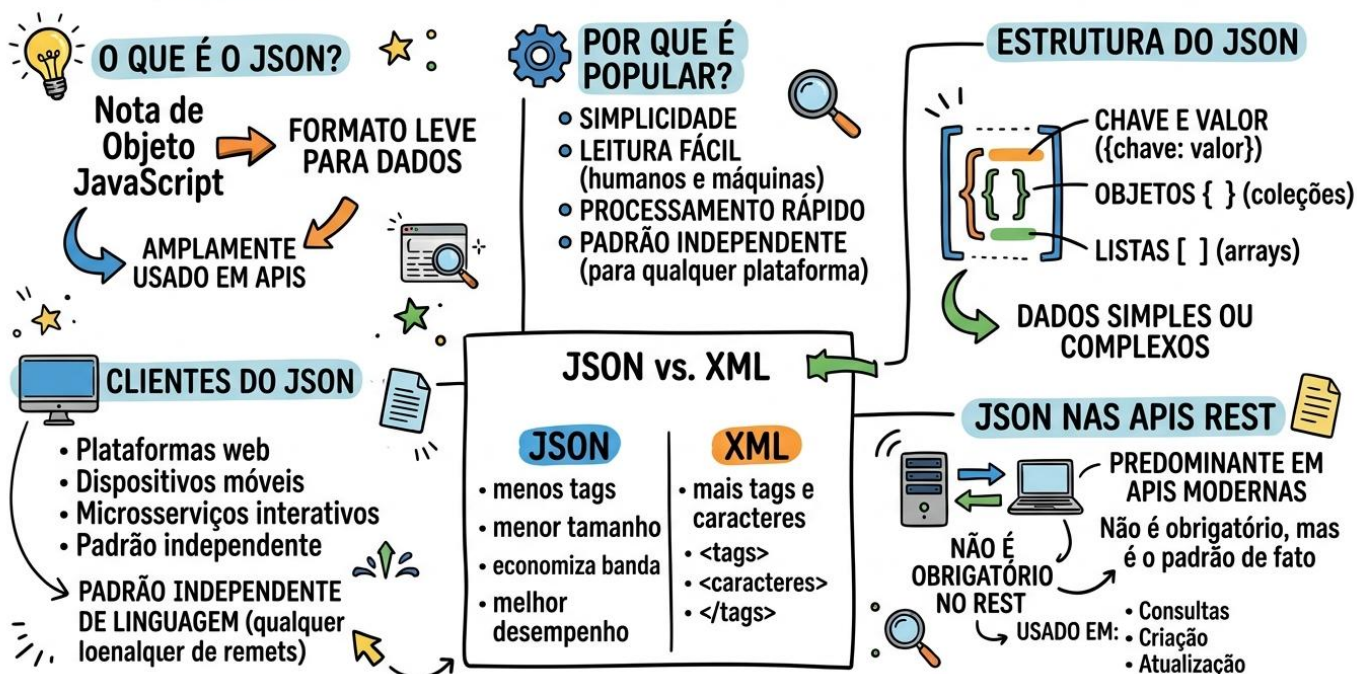
INCIDÊNCIA EM PROVA: MÉDIA

JSON

JSON, sigla para JavaScript Object Notation, é um formato leve de representação de dados amplamente utilizado em APIs REST. Sua estrutura é baseada em pares chave-valor e coleções organizadas em objetos e arrays. Por ser simples, compacto e de fácil interpretação tanto por humanos quanto por máquinas, tornou-se o formato mais popular para troca de informações em aplicações Web modernas.

JSON, JavaScript Object Notation, é um dos formatos de intercâmbio de dados mais utilizados em aplicações modernas. Qual característica explica sua ampla adoção? A combinação entre simplicidade, legibilidade e facilidade de processamento. Embora tenha surgido da sintaxe do JavaScript, JSON tornou-se um padrão independente e pode ser utilizado por praticamente qualquer plataforma. Uma das vantagens mais conhecidas está na estrutura compacta: em muitos cenários, JSON utiliza menos marcações do que XML, produzindo representações mais concisas.

JSON: O FORMATO DE DADOS MAIS POPULAR



Para representar dados, JSON utiliza elementos fundamentais: objetos, delimitados por chaves e compostos por pares chave-valor; arrays, delimitados por colchetes que armazenam coleções ordenadas de valores; e valores individuais como textos, números, booleanos, objetos, arrays ou nulo. Esses elementos podem ser combinados em diferentes níveis de profundidade, permitindo modelar desde informações simples até estruturas complexas. Essa flexibilidade explica por que o formato é utilizado em contextos tão diversos, como sistemas corporativos e APIs modernas.

Ao estudar APIs REST, é comum encontrar JSON em praticamente todos os exemplos. Isso significa que REST exige esse formato? Não. A arquitetura REST permite diferentes formas de representação dos recursos. Entretanto, JSON tornou-se a opção predominante pela simplicidade e ampla compatibilidade com linguagens e plataformas modernas. Outra característica destacada é sua legibilidade: mesmo quem

possui pouca experiência com programação consegue identificar o significado geral dos dados observando a organização das chaves e dos valores.

A utilização de chaves como identificadores explícitos de cada atributo é um dos fatores que tornam o JSON mais legível do que formatos posicionais. Em vez de interpretar sequências de dados por posição, os sistemas localizam diretamente as informações desejadas por meio de seus identificadores. Arrays são frequentemente utilizados para representar listas em APIs. A combinação entre objetos e arrays permite modelar estruturas sofisticadas sem mecanismos adicionais, o que tornou o JSON o formato predominante em serviços distribuídos modernos.

Objetos

OBJETOS

Um objeto é a principal estrutura de organização de dados no formato JSON. Ele representa uma coleção de informações relacionadas, armazenadas na forma de pares chave-valor, em que cada chave identifica um atributo e cada valor corresponde ao dado associado. Os objetos são delimitados por chaves ({}) e podem conter diferentes tipos de valores, como números, textos, valores booleanos, arrays ou até mesmo outros objetos, permitindo representar estruturas de dados simples ou complexas.

O objeto é uma das estruturas mais importantes do JSON e desempenha papel central na organização das informações. O que caracteriza um objeto nesse formato? Trata-se de uma coleção de pares chave-valor utilizada para agrupar dados relacionados dentro de um mesmo contexto. Os objetos são delimitados por chaves, e cada chave funciona como identificador de um atributo, representada como string entre aspas duplas. Os valores podem ser textos, números, booleanos, objetos, arrays ou nulo, oferecendo grande flexibilidade de representação.

Para compreender melhor essa estrutura, imagine a representação de um cliente. Atributos como nome, idade e e-mail são armazenados como pares chave-valor dentro do objeto, mantendo os dados relacionados em uma única estrutura lógica. Uma característica importante é a capacidade de composição: objetos podem conter outros objetos. Um pedido pode incluir informações próprias e também um objeto representando o cliente responsável, criando estruturas hierárquicas para descrever relacionamentos. Essa flexibilidade torna o JSON amplamente utilizado em APIs.

Arrays

ARRAYS

Um array é uma estrutura utilizada no JSON para armazenar uma sequência ordenada de valores. Diferentemente dos objetos, cujos elementos são identificados por chaves, os itens de um array são acessados por sua posição na lista. Os arrays são delimitados por colchetes ([]) e podem conter valores de diferentes tipos, como números, textos, valores booleanos, objetos e até mesmo outros arrays.

Arrays são estruturas utilizadas para armazenar coleções ordenadas de valores em documentos JSON. Sua principal finalidade é agrupar múltiplas informações em uma única estrutura de forma organizada. Os arrays são delimitados por colchetes e podem armazenar textos, números, valores booleanos, objetos, outros arrays e o valor nulo. Uma situação bastante comum envolve o retorno de listas por APIs: cada elemento pode ser representado por um objeto independente, permitindo transmitir grandes volumes de informações de maneira estruturada e processável.

Uma característica importante dos arrays está relacionada à ordem dos elementos armazenados. Cada item ocupa uma posição específica dentro da coleção, e essa sequência é preservada durante o processamento. Em cenários como históricos de eventos ou listas classificadas, a posição de cada

elemento pode possuir significado relevante para a aplicação. A combinação entre arrays e objetos amplia o poder de representação do JSON, permitindo modelar estruturas sofisticadas compostas por múltiplos níveis de relacionamento entre informações distintas.

Pares Chave-Valor

PARES CHAVE-VALOR

Os pares chave-valor constituem a forma básica de organização das informações em um objeto JSON. Cada dado é representado por uma chave, que identifica o atributo, e por um valor, que corresponde à informação armazenada nesse atributo. A chave é sempre escrita como uma cadeia de caracteres entre aspas duplas, seguida por dois-pontos (:) e pelo respectivo valor, que pode ser um número, texto, valor booleano, objeto, array ou null.

Os pares chave-valor representam o mecanismo fundamental utilizado pelo JSON para organizar informações. Cada informação é associada a uma chave, responsável por identificar o atributo, e a um valor, que contém o dado correspondente. As chaves são sempre representadas como strings delimitadas por aspas duplas, enquanto os valores podem ser textos, números, booleanos, objetos, arrays e o valor nulo. O significado de cada dado não depende de sua posição na estrutura – o próprio identificador da chave informa qual atributo está sendo representado.

Essa característica contribui para a clareza das informações transmitidas. Por que os pares chave-valor são considerados tão importantes para a legibilidade do JSON? Porque cada valor permanece explicitamente associado ao atributo que representa, reduzindo ambiguidades e facilitando a compreensão humana e o processamento pelas aplicações. Objetos JSON são formados justamente por conjuntos organizados de pares chave-valor. Compreender esse mecanismo é essencial para entender como o JSON estrutura informações em aplicações modernas e serviços em APIs.

Conceitos Avançados

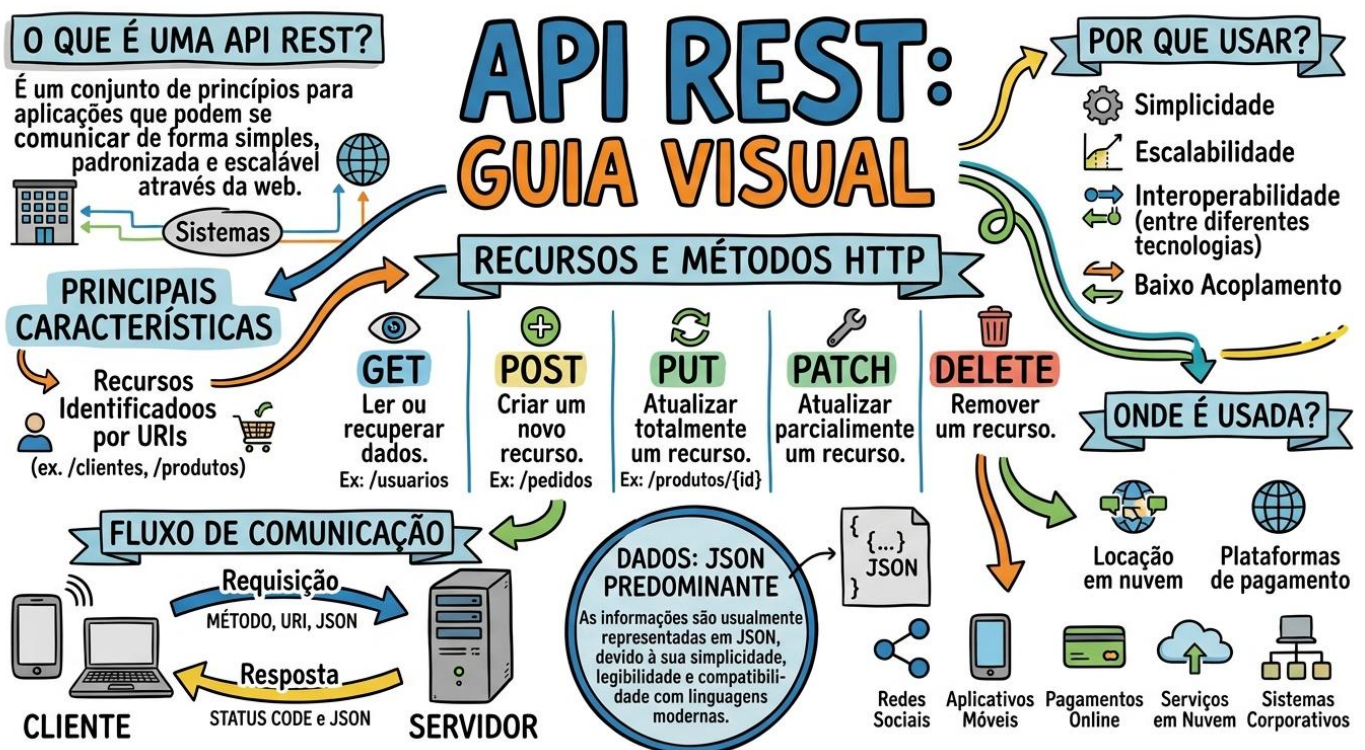
API REST

INCIDÊNCIA EM PROVA: MÉDIA

API REST

Uma API REST é uma interface de programação de aplicações desenvolvida de acordo com os princípios da arquitetura REST, permitindo que diferentes sistemas se comuniquem por meio da Web de forma simples, padronizada e escalável. Em uma API REST, os recursos são identificados por URLs e manipulados utilizando os métodos do protocolo HTTP, como GET, POST, PUT, PATCH e DELETE. As informações trocadas entre cliente e servidor são normalmente representadas no formato JSON, embora outros formatos também possam ser utilizados. Devido à sua simplicidade, flexibilidade e baixo acoplamento, as APIs REST tornaram-se o padrão predominante para integração entre aplicações Web, dispositivos móveis, microsserviços e sistemas distribuídos.

As APIs REST estão entre os mecanismos mais utilizados para integração de sistemas distribuídos. O que explica essa ampla adoção? A arquitetura foi concebida para favorecer simplicidade, interoperabilidade e escalabilidade durante a troca de informações. Baseadas nos princípios definidos por Roy Fielding, essas APIs organizam a comunicação em torno de recursos identificados por URLs, permitindo que aplicações desenvolvidas com tecnologias distintas interajam de forma padronizada, aproveitando os mecanismos já consolidados na arquitetura da Web.



As operações são indicadas por métodos como GET, POST, PUT, PATCH e DELETE, produzindo interfaces previsíveis. O formato JSON tornou-se predominante nas APIs REST por sua simplicidade, legibilidade e ampla compatibilidade, embora REST não exija formato específico de dados. A presença das APIs REST pode ser observada em praticamente todos os segmentos da tecnologia: plataformas de pagamento, sistemas corporativos, aplicações móveis e serviços em nuvem utilizam esse modelo para disponibilizar dados e funcionalidades de forma padronizada e interoperável.

QUESTÕES COMENTADAS

1. (FGV / DPE RO - 2025) Uma interface de programação de aplicativos (API, Application Programming Interface) fornece uma maneira de integrar aplicações e conectar componentes em arquiteturas de microsserviços. Uma API REST (Representational State Transfer, ou transferência de estado representacional)

- a) proporciona conformidade integrada, fazendo com que seja uma opção recomendada para casos empresariais.
- b) é um protocolo.
- c) possibilita comunicação entre aplicações através de métodos HTTP.
- d) é um conjunto de regras e diretrizes sobre como você deve criar uma API DESKTOP.
- e) implementa a linguagem de descrição de serviços web (WSDL, Web Services Description Language).

Comentários:

- (a) Errado. A ideia de "conformidade integrada" não é uma característica central de APIs REST, então isso não explica sua adoção em cenários empresariais.
- (b) Errado. REST não é tratado aqui como protocolo, mas como um estilo de construção de APIs.
- (c) Correto. APIs REST permitem a comunicação entre aplicações usando métodos HTTP, como GET, POST, PUT e DELETE.
- (d) Errado. REST não traz regras para criar API desktop; ele orienta a construção de APIs voltadas à comunicação entre sistemas.
- (e) Errado. WSDL está ligado a outra abordagem de serviços web, não ao modelo REST.

Gabarito: Letra C

2. (FGV / DPE RS - 2023) Em um ambiente tão heterogêneo como a Internet, elementos de interoperabilidade são fundamentais, e os Web Services são o modelo mais comum para o

fornecimento de serviços independentes de plataforma. Uma característica das tecnologias de Web Services é:

- a) a criação automática de um cliente Java para Web Services do tipo SOAP é possível em diversas IDEs com o simples fornecimento do endereço do descritor de serviços, que segue a sintaxe OMG-IDL;
- b) o protocolo SOAP garante a transparência para firewalls e independência de plataforma através do uso de comunicação em modo texto no formato JSON;
- c) ao lidar com um Web Service do tipo RESTful, as operações relacionadas à alteração de dados, de acordo com o padrão estabelecido, devem ser efetuadas via método POST;
- d) segundo o padrão da arquitetura REST, a obtenção de todas as entidades a partir de um Web Service do tipo RESTful ocorrerá com o acesso ao endereço de base do serviço via método GET do protocolo HTTP;
- e) por não serem capazes de manter estado, as tecnologias de Web Services não permitem acesso autenticado, trazendo fragilidade em termos de segurança.

Comentários:

- (a) Errado. Em SOAP, o descritor de serviços é o WSDL, não uma sintaxe OMG-IDL; por isso, a ideia central da alternativa foge do padrão mais comum dos Web Services.
- (b) Errado. SOAP trabalha tradicionalmente com mensagens XML. A menção a JSON não caracteriza o protocolo e compromete a justificativa apresentada na opção.
- (c) Errado. Em RESTful, alteração de dados não fica restrita ao POST. O padrão envolve outros métodos HTTP conforme a operação desejada.
- (d) Correto. Na arquitetura REST, consultar uma coleção de recursos costuma ocorrer com requisição GET no endereço base do recurso, para obter as entidades disponibilizadas pelo serviço.
- (e) Errado. Web Services podem adotar mecanismos de autenticação e segurança. O fato de não manterem estado, por si só, não impede acesso autenticado.

Gabarito: Letra D

3. (FGV / ALEMA - 2023) SOA é um padrão de arquitetura de software de baixo acoplamento e baseada em princípios de computação distribuída. Na arquitetura SOA, o UDDI define um padrão para

- a) renderização de documentos XML em navegadores web.
- b) ofuscação de informações em banco de dados relacionais.
- c) transferência de ficheiros entre computadores remotos.
- d) visibilidade, reutilização e gerenciabilidade de serviços web.
- e) conversão de endpoints em endereços da camada de enlace.

Comentários:

(a) Errado. UDDI não trata de exibição de XML em navegador; isso está mais ligado a tecnologias de apresentação e visualização de documentos.

(b) Errado. Ofuscação de dados em banco relacional não é função do UDDI, que atua no contexto de serviços web.

(c) Errado. Transferência de ficheiros entre computadores remotos envolve outros protocolos, não o padrão UDDI.

(d) Correto. O UDDI funciona como um diretório de serviços, facilitando descoberta, visibilidade, reutilização e gerenciamento de serviços web.

(e) Errado. UDDI não faz conversão de endpoints para endereços de enlace; isso não tem relação com seu papel na arquitetura SOA.

Gabarito: Letra D

4. (FGV / ALEMA - 2023) Os trechos a seguir apontam características dos Web Services, à exceção de um. Assinale-o.

- a) podem ser utilizados por outras aplicações.
- b) são autocontidos e autodescritos.
- c) são descobertos na Web por meio do protocolo SOAP.
- d) se comunicam com protocolos abertos.
- e) têm como bases o HTTP e o XML.

Comentários:

- (a) Errado. Essa é uma característica típica de Web Services, já que eles são feitos justamente para serem consumidos por outras aplicações.
- (b) Errado. Web Services costumam ser autocontidos e autodescritos, o que facilita seu uso e integração entre sistemas diferentes.
- (c) Correto. O item foge das características pedidas na questão, por isso é o gabarito. A descoberta de serviços não é apresentada aqui como traço geral do funcionamento.
- (d) Errado. A comunicação por protocolos abertos é uma marca importante dos Web Services, favorecendo interoperabilidade entre plataformas.
- (e) Errado. HTTP e XML aparecem como bases clássicas dos Web Services, especialmente nos modelos mais tradicionais de troca de dados.

Gabarito: Letra C

5. (FGV / TJDF - 2022) No âmbito de Web services, analise as afirmativas a seguir sobre a abordagem REST e o uso de tecnologias baseadas em SOAP.

I. Uma característica dos serviços Web RESTful é a capacidade de transmitir dados diretamente via HTTP.

II. As mensagens SOAP precisam ser retornadas como documentos XML.

III. Um navegador não pode armazenar em cache uma solicitação concluída por uma API SOAP.

É correto o que se afirma em:

- a) somente II;
- b) somente I e II;
- c) somente I e III;
- d) somente II e III;
- e) I, II e III.

Comentários:

(I) Correto. Em REST, a ideia é justamente aproveitar o próprio HTTP para trocar informações de forma direta, usando seus métodos e recursos nativos.

(II) Correto. No padrão SOAP, as mensagens seguem estrutura baseada em XML, então o retorno acontece nesse formato de documento.

(III) Correto. Em APIs SOAP, em regra, não se trabalha com cache no navegador da mesma forma que em requisições HTTP mais diretas, como ocorre em abordagens REST.

Gabarito: Letra E

6. (FGV / IMBEL - 2021) No contexto de Web Services e respectivos padrões, analise as afirmativas a seguir.

I. SOAP é um protocolo para troca de mensagens estruturadas baseado em XML.

II. SDL é uma linguagem baseada em HTML para descrever Web Services.

III. XML é uma linguagem utilizada para formatar e exibir páginas Web.

Está correto o que se afirma em

- a) I, somente.
- b) II, somente.
- c) I e III, somente.
- d) II e III, somente.
- e) I, II e III.

Comentários:

(I) Correto. SOAP é mesmo um padrão/protocolo de troca de mensagens estruturadas e usa XML como base, sendo bem típico no contexto de Web Services.

(II) Errado. A descrição de Web Services não é feita em HTML. O item cita SDL, mas, para esse contexto, a ideia apresentada não se sustenta.

(III) Errado. XML não é usado para exibir páginas Web. Ele serve para estruturar e transportar dados, não para apresentação visual.

Gabarito: Letra A

7. (FGV / TCE-AM - 2021) Considere as seguintes frases no contexto de Web Services.

I. Um documento que descreve um web service.

II. Um protocolo de comunicação entre aplicações que rodam em diferentes sistemas operacionais.

III. Uma forma comum para descrever informações que são lidas e entendidas por aplicações computacionais.

IV. Permite a distribuição de conteúdo atualizado para outros websites.

As frases descrevem, de forma sucinta e na ordem em que aparecem, artefatos conhecidos como:

- a) RDF, RSS, WSDL, SOAP;
- b) RSS, WSDL, SOAP, RDF;
- c) SOAP, WSDL, RDF, RSS;
- d) UDDI, SOAP, RDF, WSDL;
- e) WSDL, SOAP, RDF, RSS.

Comentários:

(a) Errado. RDF não é o documento que descreve um web service; além disso, a sequência toda não bate com as definições apresentadas no enunciado.

(b) Errado. RSS não descreve web service, e WSDL também não é protocolo de comunicação entre aplicações. A ordem dos conceitos ficou trocada.

(c) Errado. SOAP é protocolo de comunicação, não o documento descritivo do serviço. Já WSDL também não aparece na posição adequada nessa sequência.

(d) Errado. UDDI está ligado a diretórios de serviços, não à descrição direta do web service pedida no item I. Por isso, a sequência não corresponde ao enunciado.

(e) Correto. WSDL descreve o web service; SOAP faz a comunicação entre aplicações; RDF descreve informações de modo compreensível por máquinas; RSS permite distribuir conteúdo atualizado.

Gabarito: Letra E

8. (FGV / ALESC - 2024) As tecnologias Standard Object Access Protocol (SOAP) e Representational State Transfer (REST) representam duas maneiras diferentes de conectar aplicativos e definem como as Application Programming Interface (APIs, ou interfaces de programação de aplicações) são criadas, possibilitando a comunicação dos dados entre aplicações web. Considerando as diferenças entre as tecnologias SOAP e REST, pode ser identificado que

- a) SOAP é um protocolo, que oferece implementação mais rígida, e REST é um conjunto de diretrizes, que oferece uma implementação flexível.
- b) APIs REST são mais pesadas que implementações SOAP, entretanto, são ideais para contextos mais modernos.
- c) Os serviços web SOAP oferecem segurança integrada e transações em conformidade que atendem a muitas necessidades empresariais, resultando em sistemas leves.
- d) REST é um conjunto de padronizações que oferece uma implementação rígida.
- e) SOAP é um protocolo com requisitos flexíveis e dinâmicos, com implementação mais leve que o REST.

Comentários:

- (a) Correto. SOAP é um protocolo com regras bem definidas e estrutura mais rígida; já o REST funciona como um conjunto de diretrizes arquiteturais, dando mais flexibilidade na implementação das APIs.
- (b) Errado. Em geral, REST é mais leve que SOAP, justamente por usar abordagens mais simples e menos verbosas, o que combina melhor com aplicações web modernas.
- (c) Errado. SOAP realmente oferece recursos fortes de segurança e transações, muito usados no meio corporativo, mas isso não resulta em sistemas leves, e sim mais robustos e estruturados.
- (d) Errado. REST não impõe uma implementação rígida. A ideia central dele é justamente oferecer princípios e restrições que permitem soluções mais flexíveis.
- (e) Errado. SOAP não é marcado por flexibilidade e leveza maior que REST. Ele costuma exigir mais estrutura e formalismo, enquanto REST tende a ser mais simples e dinâmico.

Gabarito: Letra A

9. (FGV / CVM - 2024) O analista João desenvolveu o web service cvmWS, com base no protocolo Simple Object Access Protocol (SOAP). A fim de otimizar a serialização de um arquivo binário anexo à mensagem SOAP, João implementou em cvmWS a especificação SOAP Message Transmission Optimization Mechanism (MTOM).

Quando a serialização otimizada do MTOM é bem-sucedida, os clientes do cvmWS devem ler o conteúdo do arquivo anexo contido:

- a) dentro do envelope SOAP, no elemento xop;
- b) fora do envelope SOAP, no corpo da mensagem;
- c) dentro do envelope SOAP, no elemento binaryData;
- d) dentro do envelope SOAP, no elemento base64Binary;
- e) fora do envelope SOAP, em uma mensagem subsequente.

Comentários:

- (a) Errado. No MTOM, a ideia não é manter o anexo propriamente dito em um elemento interno do envelope SOAP, como xop, mas otimizar seu transporte de outro modo.
- (b) Correto. Com MTOM, o conteúdo binário otimizado fica fora do envelope SOAP, no corpo da mensagem transmitida, o que evita sobrecarga de serialização em Base64.
- (c) Errado. Não há leitura do anexo otimizado em um elemento chamado binaryData dentro do envelope; essa não é a forma adotada pelo MTOM para o conteúdo binário.
- (d) Errado. O uso de base64Binary representa justamente a forma não otimizada de carregar binário no XML, enquanto o MTOM busca evitar esse custo quando consegue otimizar.
- (e) Errado. O anexo otimizado não é lido em uma mensagem posterior; ele segue junto da transmissão da mensagem, só que fora do envelope SOAP.

Gabarito: Letra B

10.(FGV / SEPLAG Niterói - 2024) Um servidor está desenvolvendo um serviço para integração entre sistemas e foi exigido que a camada de transporte utilizasse SMTP. Ele recebeu um arquivo WSDL para orientá-lo sobre como utilizar o serviço. É correto afirmar que se tratava de um serviço do tipo

- a) XML.
- b) SOAP.
- c) Ajax.
- d) JSON-RPC.
- e) REST.

Comentários:

- (a) Errado. XML é uma linguagem de marcação para estruturar dados, não o tipo de serviço em si.

- (b) Correto. WSDL é usado para descrever serviços SOAP, e esse padrão pode usar SMTP como protocolo de transporte.
- (c) Errado. Ajax é uma técnica de interação assíncrona em aplicações web, não um tipo de serviço descrito por WSDL.
- (d) Errado. JSON-RPC usa JSON para chamadas remotas e não se relaciona, em regra, ao uso de WSDL nesse contexto.
- (e) Errado. REST normalmente não é descrito por WSDL da forma clássica associada aos serviços SOAP.

Gabarito: Letra B

11.(FGV / TJ RN - 2023) No contexto de uma mensagem SOAP, o elemento que NÃO faz parte da estrutura sintática estabelecida é:

- a) Body;
- b) Credential;
- c) Envelope;
- d) Fault;
- e) Header.

Comentários:

- (a) Errado. Body é um dos elementos clássicos da mensagem SOAP, usado para carregar o conteúdo principal da comunicação.
- (b) Correto. Credential não integra a estrutura sintática padrão do SOAP. Pode até aparecer em soluções específicas, mas não como elemento estrutural básico.
- (c) Errado. Envelope faz parte da base do SOAP, pois delimita e organiza toda a mensagem.
- (d) Errado. Fault é um elemento previsto no SOAP para indicar erros no processamento da mensagem.
- (e) Errado. Header também compõe a estrutura SOAP, reunindo informações adicionais da mensagem.

Gabarito: Letra B

12.(FGV / TCE SP - 2023) Roberta está desenvolvendo um determinado serviço na rede. Foi exigida a adoção de JMS na camada de transporte, e ela recebeu um arquivo WSDL para instruí-la acerca da utilização do serviço. Roberta logo notou que se tratava de um serviço do tipo:

- a) REST;
- b) GraphQL;
- c) gRPC;
- d) Web Socket;
- e) SOAP.

Comentários:

(a) Errado. REST costuma trabalhar com recursos e verbos HTTP, sem depender de WSDL como elemento central de descrição do serviço.

(b) Errado. GraphQL usa uma abordagem baseada em schema e consultas flexíveis, não sendo identificado pelo uso de WSDL.

(c) Errado. gRPC é associado a contratos em Protocol Buffers, não a arquivos WSDL para orientar o consumo do serviço.

(d) Errado. Web Socket é voltado à comunicação persistente e bidirecional, sem essa característica típica de descrição por WSDL.

(e) Correto. WSDL é uma marca bem clássica de serviços SOAP, pois descreve operações, mensagens e formas de acesso do serviço.

Gabarito: Letra E

13.(FGV / TJDF - 2022) O DataJud é uma base composta por dados e metadados processuais relativos a todos os processos físicos ou eletrônicos, públicos ou sigilosos dos tribunais. Para alimentar o DataJud via API REST com os metadados dos processos do TJDF, Pedro recebeu o arquivo1 e enviou o arquivo2, cujos recortes estão ilustrados a seguir.

Recorte do arquivo1:

```
<attribute name="codigoDocumento" type="string"
use="required">
<annotation>
```

```
<documentation> Sequência de caracteres que tornam único o documento descrito sem os
separadores característicos.</documentation>
```

```

</annotation>
</attribute>
<attribute name="emissorDocumento" type="string" use="required">
<annotation>
<documentation> Órgão emissor do documento de identificação.</documentation>
</annotation>
</attribute>
<attribute name="tipoDocumento" type="cnj:modalidadeDocumentoidentificador"
use="required">
<annotation>
<documentation>Tipo do documento, conforme emuneração existente no tipo
"tipoDocumento".
</documentation>
</annotation>
</attribute>

```

Recorte do arquivo2:

```

<documento codigoDocumento="00104875732"
emissorDocumento="SSP" tipoDocumento="NB"/>

```

Os arquivo1 e arquivo2 utilizados por Pedro são, respectivamente:

- a) modelo de transmissão ou transferência de dados em XSD e metadados processuais em XML;
- b) metadados processuais em XSD e dados processuais em XML;
- c) Tabelas Processuais Unificadas em XML e conteúdo JSON com o XML processado;
- d) API REST de envio e arquivo XML processado pelo Validador;
- e) template de metadados processuais em XML e dados processuais em XML.

Comentários:

(a) Correto. O arquivo1 traz a estrutura e as definições dos campos, bem no papel de um modelo XSD; já o arquivo2 corresponde aos metadados processuais enviados em XML pela API REST.

(b) Errado. Aqui os papéis foram invertidos: o XSD funciona como modelo de validação/estrutura, enquanto o XML é o arquivo com os metadados efetivamente transmitidos.

(c) Errado. O enunciado fala em alimentação do DataJud via API REST com metadados processuais, não em Tabelas Processuais Unificadas nem em conteúdo JSON com XML processado.

(d) Errado. API REST não é o arquivo1, e o enunciado também não indica arquivo XML processado por validador como sendo o arquivo2.

(e) Errado. O arquivo1 não é um template em XML, mas sim o modelo estrutural dos dados, associado ao XSD; o arquivo2 é que representa os metadados em XML.

Gabarito: Letra A

14.(FGV / TJDF - 2022) O analista de sistemas Bravo desenvolveu o webservice TJService para o TJDF sem mecanismos de controle de segurança. A comunicação de TJService ocorre conforme especificações do protocolo SOAP (Simple Object Access Protocol). A fim de proteger a troca de mensagens de TJService, Bravo foi orientado pelo TJDF a implementar o suporte à autenticação baseada em token.

Os mecanismos de proteção de mensagens SOAP baseados em token que Bravo deve implementar são definidos na especificação Web Services:

- a) Dynamic Discovery;
- b) Addressing;
- c) Reliable Messaging;
- d) Security;
- e) Make Connection.

Comentários:

(a) Errado. Dynamic Discovery trata de descoberta dinâmica de serviços, não de proteção de mensagens SOAP com autenticação por token.

(b) Errado. Addressing serve para endereçamento das mensagens entre serviços, não para implementar mecanismos de autenticação.

(c) Errado. Reliable Messaging cuida da entrega confiável das mensagens, garantindo envio e recebimento, não da segurança.

(d) Correto. Security é a especificação voltada à proteção de mensagens SOAP, incluindo autenticação baseada em token.

(e) Errado. Make Connection está ligado ao estabelecimento/manutenção da comunicação, não aos mecanismos de segurança por token.

Gabarito: Letra D

15.(FGV / MPE SC - 2022) A analista em TI Isabel está implementando o web service DocService para o MPSC. A comunicação do DocService deve se basear em especificações derivadas do protocolo SOAP (Simple Object Access Protocol) e do framework WS-Security (Web Services Security). Para que DocService possa utilizar contextos de segurança baseados em token entre múltiplas mensagens SOAP, Isabel deve observar diretamente a especificação WS (Web Services):

- a) BrokeredNotification;
- b) MakeConnection;
- c) SecureConversation;
- d) BaseNotification;
- e) MetadataExchange.

Comentários:

(a) Errado. BrokeredNotification está ligado a mecanismos de notificação em web services, não ao uso de contexto de segurança com token entre várias mensagens SOAP.

(b) Errado. MakeConnection trata de aspectos de troca de mensagens e conectividade, não da manutenção de contexto de segurança entre mensagens.

(c) Correto. SecureConversation é a especificação voltada justamente para estabelecer e reutilizar contextos de segurança baseados em token em múltiplas mensagens SOAP.

(d) Errado. BaseNotification se relaciona a notificações e eventos, não ao gerenciamento de contexto de segurança em conversas SOAP.

(e) Errado. MetadataExchange serve para troca de metadados do serviço, como descrições e políticas, e não para contexto de segurança baseado em token.

Gabarito: Letra C

LISTA DE QUESTÕES

1. (FGV / DPE RO - 2025) Uma interface de programação de aplicativos (API, Application Programming Interface) fornece uma maneira de integrar aplicações e conectar componentes em arquiteturas de microsserviços. Uma API REST (Representational State Transfer, ou transferência de estado representacional)

- a) proporciona conformidade integrada, fazendo com que seja uma opção recomendada para casos empresariais.
- b) é um protocolo.
- c) possibilita comunicação entre aplicações através de métodos HTTP.
- d) é um conjunto de regras e diretrizes sobre como você deve criar uma API DESKTOP.
- e) implementa a linguagem de descrição de serviços web (WSDL, Web Services Description Language).

2. (FGV / DPE RS - 2023) Em um ambiente tão heterogêneo como a Internet, elementos de interoperabilidade são fundamentais, e os Web Services são o modelo mais comum para o fornecimento de serviços independentes de plataforma. Uma característica das tecnologias de Web Services é:

- a) a criação automática de um cliente Java para Web Services do tipo SOAP é possível em diversas IDEs com o simples fornecimento do endereço do descritor de serviços, que segue a sintaxe OMG-IDL;
- b) o protocolo SOAP garante a transparência para firewalls e independência de plataforma através do uso de comunicação em modo texto no formato JSON;
- c) ao lidar com um Web Service do tipo RESTful, as operações relacionadas à alteração de dados, de acordo com o padrão estabelecido, devem ser efetuadas via método POST;
- d) segundo o padrão da arquitetura REST, a obtenção de todas as entidades a partir de um Web Service do tipo RESTful ocorrerá com o acesso ao endereço de base do serviço via método GET do protocolo HTTP;
- e) por não serem capazes de manter estado, as tecnologias de Web Services não permitem acesso autenticado, trazendo fragilidade em termos de segurança.

3. (FGV / ALEMA - 2023) SOA é um padrão de arquitetura de software de baixo acoplamento e baseada em princípios de computação distribuída. Na arquitetura SOA, o UDDI define um padrão para

- a) renderização de documentos XML em navegadores web.
- b) ofuscação de informações em banco de dados relacionais.
- c) transferência de ficheiros entre computadores remotos.
- d) visibilidade, reutilização e gerenciabilidade de serviços web.
- e) conversão de endpoints em endereços da camada de enlace.

4. (FGV / ALEMA - 2023) Os trechos a seguir apontam características dos Web Services, à exceção de um. Assinale-o.

- a) podem ser utilizados por outras aplicações.
- b) são autocontidos e autodescritos.
- c) são descobertos na Web por meio do protocolo SOAP.
- d) se comunicam com protocolos abertos.
- e) têm como bases o HTTP e o XML.

5. (FGV / TJDF - 2022) No âmbito de Web services, analise as afirmativas a seguir sobre a abordagem REST e o uso de tecnologias baseadas em SOAP.

I. Uma característica dos serviços Web RESTful é a capacidade de transmitir dados diretamente via HTTP.

II. As mensagens SOAP precisam ser retornadas como documentos XML.

III. Um navegador não pode armazenar em cache uma solicitação concluída por uma API SOAP.

É correto o que se afirma em:

- a) somente II;
- b) somente I e II;
- c) somente I e III;
- d) somente II e III;
- e) I, II e III.

6. (FGV / IMBEL - 2021) No contexto de Web Services e respectivos padrões, analise as afirmativas a seguir.

I. SOAP é um protocolo para troca de mensagens estruturadas baseado em XML.

II. SDL é uma linguagem baseada em HTML para descrever Web Services.

III. XML é uma linguagem utilizada para formatar e exibir páginas Web.

Está correto o que se afirma em

- a) I, somente.
- b) II, somente.
- c) I e III, somente.
- d) II e III, somente.
- e) I, II e III.

7. (FGV / TCE-AM - 2021) Considere as seguintes frases no contexto de Web Services.

I. Um documento que descreve um web service.

II. Um protocolo de comunicação entre aplicações que rodam em diferentes sistemas operacionais.

III. Uma forma comum para descrever informações que são lidas e entendidas por aplicações computacionais.

IV. Permite a distribuição de conteúdo atualizado para outros websites.

As frases descrevem, de forma sucinta e na ordem em que aparecem, artefatos conhecidos como:

- a) RDF, RSS, WSDL, SOAP;
- b) RSS, WSDL, SOAP, RDF;
- c) SOAP, WSDL, RDF, RSS;
- d) UDDI, SOAP, RDF, WSDL;
- e) WSDL, SOAP, RDF, RSS.

8. (FGV / ALESC - 2024) As tecnologias Standard Object Access Protocol (SOAP) e Representational State Transfer (REST) representam duas maneiras diferentes de conectar aplicativos e definem como as Application Programming Interface (APIs, ou interfaces de programação de aplicações) são criadas, possibilitando a comunicação dos dados entre aplicações web. Considerando as diferenças entre as tecnologias SOAP e REST, pode ser identificado que

- a) SOAP é um protocolo, que oferece implementação mais rígida, e REST é um conjunto de diretrizes, que oferece uma implementação flexível.
- b) APIs REST são mais pesadas que implementações SOAP, entretanto, são ideais para contextos mais modernos.

- c) Os serviços web SOAP oferecem segurança integrada e transações em conformidade que atendem a muitas necessidades empresariais, resultando em sistemas leves.
- d) REST é um conjunto de padronizações que oferece uma implementação rígida.
- e) SOAP é um protocolo com requisitos flexíveis e dinâmicos, com implementação mais leve que o REST.

9. (FGV / CVM - 2024) O analista João desenvolveu o web service cvmWS, com base no protocolo Simple Object Access Protocol (SOAP). A fim de otimizar a serialização de um arquivo binário anexo à mensagem SOAP, João implementou em cvmWS a especificação SOAP Message Transmission Optimization Mechanism (MTOM).

Quando a serialização otimizada do MTOM é bem-sucedida, os clientes do cvmWS devem ler o conteúdo do arquivo anexo contido:

- a) dentro do envelope SOAP, no elemento xop;
- b) fora do envelope SOAP, no corpo da mensagem;
- c) dentro do envelope SOAP, no elemento binaryData;
- d) dentro do envelope SOAP, no elemento base64Binary;
- e) fora do envelope SOAP, em uma mensagem subsequente.

10.(FGV / SEPLAG Niterói - 2024) Um servidor está desenvolvendo um serviço para integração entre sistemas e foi exigido que a camada de transporte utilizasse SMTP. Ele recebeu um arquivo WSDL para orientá-lo sobre como utilizar o serviço. É correto afirmar que se tratava de um serviço do tipo

- a) XML.
- b) SOAP.
- c) Ajax.
- d) JSON-RPC.
- e) REST.

11.(FGV / TJ RN - 2023) No contexto de uma mensagem SOAP, o elemento que NÃO faz parte da estrutura sintática estabelecida é:

- a) Body;
- b) Credential;
- c) Envelope;
- d) Fault;
- e) Header.

12.(FGV / TCE SP - 2023) Roberta está desenvolvendo um determinado serviço na rede. Foi exigida a adoção de JMS na camada de transporte, e ela recebeu um arquivo WSDL para instruí-la acerca da utilização do serviço. Roberta logo notou que se tratava de um serviço do tipo:

- a) REST;
- b) GraphQL;
- c) gRPC;
- d) Web Socket;
- e) SOAP.

13.(FGV / TJDF - 2022) O DataJud é uma base composta por dados e metadados processuais relativos a todos os processos físicos ou eletrônicos, públicos ou sigilosos dos tribunais. Para alimentar o DataJud via API REST com os metadados dos processos do TJDF, Pedro recebeu o arquivo1 e enviou o arquivo2, cujos recortes estão ilustrados a seguir.

Recorte do arquivo1:

```
<attribute name="codigoDocumento" type="string"
use="required">
<annotation>
<documentation> Sequência de caracteres que tornam único o documento descrito sem os
separadores característicos.</documentation>
</annotation>
</attribute>
<attribute name="emissorDocumento" type="string" use="required">
<annotation>
<documentation> Órgão emissor do documento de identificação.</documentation>
</annotation>
</attribute>
<attribute name="tipoDocumento" type="cnj:modalidadeDocumentoIdentificador"
use="required">
<annotation>
<documentation>Tipo do documento, conforme emuneração existente no tipo
"tipoDocumento".
</documentation>
</annotation>
</attribute>
```

Recorte do arquivo2:

```
<documento codigoDocumento="00104875732"
```

emissorDocumento="SSP" tipoDocumento="NB"/>

Os arquivo1 e arquivo2 utilizados por Pedro são, respectivamente:

- a) modelo de transmissão ou transferência de dados em XSD e metadados processuais em XML;
- b) metadados processuais em XSD e dados processuais em XML;
- c) Tabelas Processuais Unificadas em XML e conteúdo JSON com o XML processado;
- d) API REST de envio e arquivo XML processado pelo Validador;
- e) template de metadados processuais em XML e dados processuais em XML.

14.(FGV / TJDF - 2022) O analista de sistemas Bravo desenvolveu o webservice TJService para o TJDF sem mecanismos de controle de segurança. A comunicação de TJService ocorre conforme especificações do protocolo SOAP (Simple Object Access Protocol). A fim de proteger a troca de mensagens de TJService, Bravo foi orientado pelo TJDF a implementar o suporte à autenticação baseada em token.

Os mecanismos de proteção de mensagens SOAP baseados em token que Bravo deve implementar são definidos na especificação Web Services:

- a) Dynamic Discovery;
- b) Addressing;
- c) Reliable Messaging;
- d) Security;
- e) Make Connection.

15.(FGV / MPE SC - 2022) A analista em TI Isabel está implementando o web service DocService para o MPSC. A comunicação do DocService deve se basear em especificações derivadas do protocolo SOAP (Simple Object Access Protocol) e do framework WS-Security (Web Services Security). Para que DocService possa utilizar contextos de segurança baseados em token entre múltiplas mensagens SOAP, Isabel deve observar diretamente a especificação WS (Web Services):

- a) BrokeredNotification;
- b) MakeConnection;
- c) SecureConversation;
- d) BaseNotification;
- e) MetadataExchange.

GABARITO

- | | | | | | |
|----|---------|-----|---------|-----|---------|
| 1. | LETRA C | 6. | LETRA A | 11. | LETRA B |
| 2. | LETRA D | 7. | LETRA E | 12. | LETRA E |
| 3. | LETRA D | 8. | LETRA A | 13. | LETRA A |
| 4. | LETRA C | 9. | LETRA B | 14. | LETRA D |
| 5. | LETRA E | 10. | LETRA B | 15. | LETRA C |

ESSA LEI TODO MUNDO CONHECE: PIRATARIA É CRIME.

Mas é sempre bom revisar o porquê e como você pode ser prejudicado com essa prática.



1 Professor investe seu tempo para elaborar os cursos e o site os coloca à venda.



2 Pirata divulga ilicitamente (grupos de rateio), utilizando-se do anonimato, nomes falsos ou laranjas (geralmente o pirata se anuncia como formador de "grupos solidários" de rateio que não visam lucro).



3 Pirata cria alunos fake praticando falsidade ideológica, comprando cursos do site em nome de pessoas aleatórias (usando nome, CPF, endereço e telefone de terceiros sem autorização).



4 Pirata compra, muitas vezes, clonando cartões de crédito (por vezes o sistema anti-fraude não consegue identificar o golpe a tempo).



5 Pirata fere os Termos de Uso, adultera as aulas e retira a identificação dos arquivos PDF (justamente porque a atividade é ilegal e ele não quer que seus fakes sejam identificados).



6 Pirata revende as aulas protegidas por direitos autorais, praticando concorrência desleal e em flagrante desrespeito à Lei de Direitos Autorais (Lei 9.610/98).



7 Concurseiro(a) desinformado participa de rateio, achando que nada disso está acontecendo e esperando se tornar servidor público para exigir o cumprimento das leis.



8 O professor que elaborou o curso não ganha nada, o site não recebe nada, e a pessoa que praticou todos os ilícitos anteriores (pirata) fica com o lucro.



Deixando de lado esse mar de sujeira, aproveitamos para agradecer a todos que adquirem os cursos honestamente e permitem que o site continue existindo.